Corralling AI Risks with Grassroots Contracts (Full Version)

Ehud Shapiro
London School of Economics and Weizmann Institute of Science
UK and Israel

ABSTRACT

AI's inflection point is a fountain of risks, with digital platforms in particular being vulnerable, as those in control of global platforms cede control of the algorithms, membership and content to AI:

(1) **AI control of global platforms**—social and economic—used by the majority of humanity, resulting in the majority of humanity becoming vulnerable to AI control.
(2) **Unaccountable AI agents** operating anonymously on social and economic digital platforms, side-by-side with humans, possibly masquerading as humans, serving unknown masters with unknown goals, operating under unknown restraints without accountability or culpability.
(3) **Unattributed deepfakes** generated by anonymous AI, with people unable to distinguish them from genuine content and with their producers evading accountability and culpability.

Here, we propose to corral these AI risks with grassroots contracts—a digital incarnation of a two-centuries old notion of Pierre-Joseph Proudhon. Proudhon envisioned a social contract that can be freely-adopted by any group of people, where groups bound by the same social contract can union by mutual consent, possibly into a single global union. Grassroots contracts are realised by grassroots platforms—smartphone-based person-to-person digital platforms owned, operated and governed by their participants. Fundamental grassroots platforms include the grassroots social graph, grassroots social networks, child-safe social networks, and grassroots currencies, with higher-level platforms providing for democratic digital communities and "sharing-economy" cooperatives. We developed, with AI, a methodology to compile grassroots contracts to working smartphone apps. We present grassroots contracts for a social graph with accountable AI agents and for a social network with digital provenance, and working smartphone apps that implement the two and show how they corral these major AI risks:

(1) **AI control of global platforms** is mitigated by grassroots platforms by design, as, unlike global platforms, they are not controlled by a third party who may then cede control to AI.
(2) **Unaccountable AI agents** are excluded, as AI agents are introduced to the grassroots social graph with accountability: similarly to cars, their manufacturers are known and are accountable for manufacturing defects, and their operators are known and are accountable for reckless operation.
(3) **Unattributed deepfakes** cannot circulate in grassroots social networks: every item carries the signatures of its author and of every agent that forwarded it, so creators and forwarders of harmful content are accountable and culpable.

Grassroots platforms are presently a concept with mathematical and computational theory, working prototypes, and tools for their large-scale simulation and testing. Much work remains for grassroots smartphone apps to be competitive with incumbent global platforms; but perhaps now this could be achieved with some AI-months rather than with incalculably-many person-years.

CCS CONCEPTS

• **Computing methodologies** → **Artificial intelligence**; **Multi-agent systems**; • **Computer systems organization** → **Peer-to-peer architectures**; • **Human-centered computing** → *Social networking sites*; • **Security and privacy** → *Digital signatures*; • **Applied computing** → *Law*; • **Social and professional topics** → *Computing / technology policy*; • **Theory of computation** → Distributed computing models.

KEYWORDS

Grassroots Platforms, Grassroots Contracts, Social Contracts, AI Risks, AI Safety, AI Agents, Accountable AI, Deepfakes, Digital Provenance, Digital Speech Acts, Genuine Digital Identity, Social Networks, Smartphone Apps

TL;DR

Those who own and control the Web's social and economic platforms are now ceding their control to AI: of the algorithms, of who may join, and of what circulates. Hence, the majority of humanity now faces AI control of global platforms, unaccountable AI agents posing as people, and unattributed deepfakes. We propose a Web owned, operated and governed by the people who use it, with no "others" who control it and could cede their control to AI; AI agents join only under operators accountable for their operation and developers accountable for their defects; and every circulating item carries digital provenance, identifying its author and those who forwarded it, so a deepfake loses the protection of anonymity by having someone accountable for it.

1 INTRODUCTION

AI's inflection point is a fountain of alarming risks [2, 3, 20, 52, 56]. Digital platforms in particular are vulnerable to three major AI risks. Those in control of global platforms are ceding control to AI to maximise the value extracted from their users: of the algorithms, amplifying commercial surveillance [59]; of membership, by letting AI agents masquerade as people [15, 31]; and of content, by letting AI generate deepfakes with no one held accountable [28].

(1) **AI control of global platforms**—social and economic—used by the majority of humanity, resulting in the majority of humanity becoming vulnerable to AI control. Platform owners may delegate the operation of their platforms to AI, giving AI systems power over the social and economic lives of their participants and compounding existing concentrations of power, surveillance and manipulation [3, 10, 51, 58]; the participants are then subject to AI control even when the AI faithfully pursues its owners' objectives, and the owners may later be unable

to reclaim the control they delegated [2]. Kulveit et al. [24] analyse the broader possibility that successive decisions to adopt AI gradually erode human influence over societal systems.

(2) **Unaccountable AI agents** operating anonymously on social and economic digital platforms, side-by-side with humans, possibly masquerading as humans, serving unknown masters with unknown goals, operating under unknown restraints without accountability or culpability. Social bots already take part in social networks and impersonate people [15, 31]. Proposals to establish, privately, that a participant is a person [1] and to make an AI agent's activity attributable through identifiers, monitoring and logs [7, 51] address two different questions: whether a person is participating, and who authorised an agent's acts and is answerable for them. Attribution must also be related to actual authority and control [34]: an identifiable person may otherwise be made to bear responsibility for a system over which they had little control, Elish's moral crumple zone [13].

(3) **Unattributed deepfakes** generated by anonymous AI, with people unable to distinguish them from genuine content and with their producers evading accountability and culpability. Untrained observers distinguish synthetic faces from real ones at about chance level [30], and a deceptive political deepfake raises uncertainty and lowers trust in news on social media [55]; Mirsky and Lee survey the creation and detection of deepfakes [28], and Chesney and Citron their consequences for privacy, democracy and national security [9]. Detection estimates whether content is synthetic; provenance records who produced, endorsed or forwarded it [8], and supports accountability without establishing that the content is true [8, 18].

Two further AI risks are not addressed here: harms to people using AI, through erroneous advice, manipulation, overreliance or emotional dependence [2, 14, 16, 56], and harms people cause with abilities amplified by AI, such as fraud, harassment and cyberattacks [2, 5, 27].

Here, we propose to corral these AI risks with grassroots contracts [6, 17], a digital incarnation of a two-centuries old notion of Pierre-Joseph Proudhon [33, Sixth Study]. Proudhon envisioned a social contract that can be freely adopted by any group of people—one could "make a contract with all, as … with some"—where groups bound by the same social contract can union by mutual consent, possibly into a single global union—"each group of citizens … formed by a like contract … could thereafter, and always by a similar contract, agree with every and all other groups". Grassroots contracts are realised by grassroots platforms [26, 38, 40, 45]: smartphone-based person-to-person digital platforms owned, operated and governed by their participants. A grassroots platform is constituted from its participants' smartphones and the grassroots app each participant runs, and requires no global resource but the network: no hardwired servers, no global authority, and nobody's permission to start. Global platforms, centralised like Facebook or decentralised like Bitcoin and Ethereum [29, 57], can have only a single instance, as two instances would clash over the global resources—domain name, port number, or boot nodes—hardwired into their code, and from the perspective of ordinary people all of them are monolithic entities owned, operated and governed by others [41]. Fundamental grassroots platforms include the grassroots social graph [12, 17, 39], grassroots social networks [39], child-safe social networks [43], and grassroots currencies [25, 37, 44, 48], with higher-level platforms providing for democratic digital communities [19, 21, 50, 53] and "sharing-economy" cooperatives (in preparation).

We developed, with AI, a methodology to compile grassroots contracts to working smartphone apps: an abstraction cascade from the contract to the app (Section 2), its correctness proved from the transactions down to GLP and the app built and tested, each layer coded by AI from the paper that describes it [11, 17, 22, 26, 42, 46, 49].

In this paper, we present grassroots contracts for a social graph with accountable AI agents (Section 3) and for a social network with digital provenance (Section 4), and working smartphone apps that implement the two (Figures 4 and 5), and show how they corral these major AI risks:

(1) **AI control of global platforms** is mitigated by grassroots platforms by design, as, unlike global platforms, they are not controlled by a third party who may then cede control to AI. A grassroots platform has no servers, no operators and no global resources, so there is no part of it whose capture is control of the platform, and to control it an AI must control the smartphones of its participants, one by one [38, 40, 41].

(2) **Unaccountable AI agents** are excluded, as AI agents are introduced to the grassroots social graph with accountability: similarly to cars, their manufacturers are known and are accountable for manufacturing defects, and their operators are known and are accountable for reckless operation. A person takes part in a grassroots platform only with agents they met or were introduced to by an agent who knows them [17, 32, 35, 39], so an AI agent joins only under a person, its principal, who attests that it runs code signed by its developer; every guardian of the AI agent is accountable for operating it recklessly, and the developer for its defects; every act of the AI agent names its principal and its developer, carries the attestation, and is signed and non-repudiable [6, 12, 18]. The contract shares its core with the child-safe social network [43], whose wards are children and whose guardians are their parents.

(3) **Unattributed deepfakes** cannot circulate in grassroots social networks: every item is a digital speech act [18], signed by its author and carried along the edges of the social graph [17, 39], and signed over by each agent that forwards it; so creators and forwarders of harmful content are accountable and culpable, and people can tell content with provenance from content without it. Some clauses of a grassroots contract are enforced by a correct protocol, and others leave non-repudiable evidence for recourse when breached [17].

Grassroots platforms are presently a concept with mathematical and computational theory [17, 26, 38, 41, 45], working prototypes [11, 22, 46], and tools for their large-scale simulation and testing [47]. We have developed grassroots theory for a decade but could not muster the resources needed to turn it into competitive software, given the not-for-profit, common-good nature of our project. Now, AI is piercing through this conundrum: our scientific papers serve as the evolving interface between us humans and AI, which codes from them [46, 49]. Much work remains for grassroots smartphone apps to be competitive with incumbent

global platforms; but perhaps now this could be achieved with some AI-months rather than with incalculably-many person-years.

Section 2 explains how a grassroots contract becomes an app, and presents the grassroots social graph and the grassroots social network with their apps. Section 3 presents the contract with accountable AI agents and Section 4 the contract with digital provenance, each with what it excludes and what it makes answerable. Section 5 reviews related work and Section 6 concludes. Appendix A recalls the framework, Appendix B lists the speech-act schemas of the two contracts, and Appendix C proves that the protocols realising them are volitionally grassroots, by the theory of contracts with wards of [17].

2 GRASSROOTS CONTRACTS

A grassroots contract [6, 17] is a few clauses in plain language, each describing an act—who takes it, who must agree to it, and what it changes—or binding the agents without describing one. Befriending is an act of two persons, both of whom must agree; ending a friendship is an act of one. The contract is the whole of the platform's design: there is no server, no operator and no account, only the persons who adopted the contract and their smartphones [38, 40]. Each clause opens with its legal modality [17]: ■ *enforced*: no act of agents can breach it; ◆ *attested* by the signatures of the actors: an agent may breach it, and the contract records the breach; ○ *undertaken* by the actors: its performance lies outside the contract, which records only the undertaking. A contract ends with its *pragmas*, which the app built from it follows: the first line names a tab of the app and what it lists, a row showing the atoms of the state naming its item, and each other line names a clause by its title, then the agent proposing, a button for each act they propose, bearing its act's name, and any field they fill, and a line for each other agent asked, their name and the question put to them [17, 22]. We show the pragmas of GAI (Section 3) and GSNP (Section 4); the pragmas of every contract are in the repository https://github.com/EShapiro2/GLP, and GKK's, GSG's and GSN's in [17].

From the contract to the app. The clauses are written as speech-act schemas [17], which name the roles of an act, which of them must will it, and what each role's state must hold before and comes to hold after; the schemas compile into volition-guarded multiagent atomic transactions [26], each a change to the states of its participants that occurs only if the persons guarding it are willing; the transactions are realised by communicating volitional agents, whose only acts between two agents are discovery and message delivery, and then by a volition-guarded Grassroots Logic Program (vGLP) [22], which is transformed into the platform's user interface and compiled to Grassroots Logic Programs (GLP) [42, 49], implemented in Dart and Flutter [46] and run as a mini-app of the Grassroots Super-App on a person's smartphone [11] (Figure 1). Correctness is proved from the transactions down to GLP [22, 26]; the app is built and tested. Each layer is coded by AI from the paper that describes it; a gap found in coding is repaired in the paper first, and the code follows [46].

What a person sees. Volition elicitation is the rationale for the user interface design [22]. An act a person must will appears on their phone as a question, with the answers the contract allows [22, 26]: Bob is asked whether to be Alice's friend, and answers Accept or Decline. A person's phone holds their own state and nothing else [38–40], their friends, their groups, the messages they sent and received, and draws what it shows from that state, by widgets the grassroots app declares for the platform's questions and for the person's state [11, 22]. A person's phone talks to the phones of the agents they know or come to know, and to nobody else [11, 12, 39]. The screens of Figures 2 and 3 are screenshots of the GSG and GSN mini-apps, and those of Figures 4 and 5 of the GSNP mini-app, each compiled from its grassroots contract down the abstraction cascade [17, 22, 26, 42, 46, 49]. One instance of the grassroots app [11] on an iPhone simulator runs the mini-app of every agent [11, 46]; the agents communicate over channels within it [42], and a screenshot shows one person's screen. All source code and all artefacts, including the grassroots contracts and the artefacts the abstraction cascade produces from them, are in the public repository https://github.com/EShapiro2/GLP.

2.1 Grassroots contracts, platforms and apps

Almost two centuries ago Pierre-Joseph Proudhon envisioned a social contract [33, Fourth Study] that is (1) an agreement of man with man; (2) reciprocal; (3) imposing no obligation upon the parties except that which results from their personal promise; (4) subject to no external authority; (5) freely accepted and signed by all the participants; and (6) of the nature of a contract of exchange. *Grassroots contracts* are social contracts meeting these six conditions and two more: (7) people are free to deal with each other; and (8) there is no external register of people, so people can come to know each other either directly or through people they already know [17].

A *digital social contract* is one realised by software the parties run [6]. We call the mathematical object a *protocol* and the software implementing it a *platform*. A protocol is *grassroots* when its instances are independent of one another and of every global resource but the net, and can always coalesce, possibly into a single global instance [38, 40, 41], and a *grassroots platform* is the software implementing a grassroots protocol [17].

A grassroots platform is constituted from its participants' smartphones and the grassroots app each participant runs [38, 40]; a person runs one app, and each platform is a mini-app of it [22]. Participating in a grassroots platform requires adopting the grassroots contract that defines it, as participating in a global platform requires adopting an End-User Licence Agreement (EULA). A grassroots contract is mutual and symmetric among all participants; an EULA is between each person and the platform operator. To join a grassroots platform, a person signs a one-sided undertaking to keep its grassroots contract towards all its other participants, and presents it to every participant they interact with, so every participant has signed it [17].

2.2 The grassroots social graph

A *person* is a legally culpable entity: a natural person of age, also referred to as an *adult*. A person takes the pronoun 'they'. An *agent* is a person, a *child*, a natural person not of age, or an *AI agent*, a program that holds a key pair of its own and takes acts of the contract with it, and takes the pronoun 'it'; this paper treats AI agents, and the child-safe social network [43] children. Every contract extends GKK, the base contract: a person establishes their

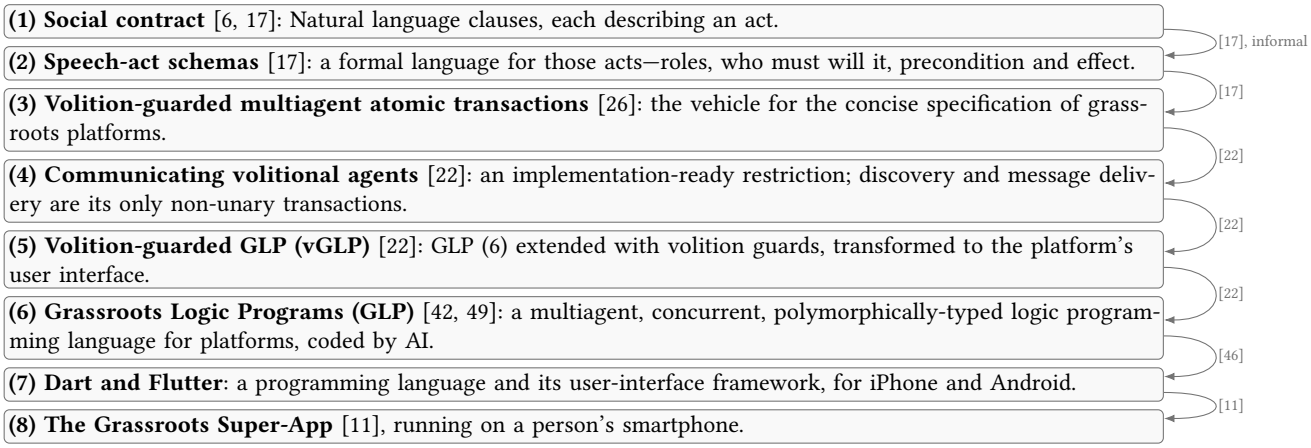


Figure 1: The abstraction cascade, from a grassroots contract to a running app. Each arrow but the first, an informal translation, represents an implementation, with the reference defining it.

identity by choosing a key pair, and *knows* another agent when they hold that agent's public key [17].

GKK: Grassroots Keys and Knowledge [17]

Alice, Bob and Carol are persons.

1. ○ *Identity.* Alice may establish her identity once by choosing a unique key pair. She makes herself known by the public key, signs her acts with the private key, and is accountable for every act signed with it.
2. ■ *Acquaintance.* Alice and Bob may come to know each other by an act of both.
3. ■ *Introduction.* If Alice and Bob know each other and Alice knows Carol, then Alice can make Carol known to Bob, with Bob's consent.

A contract that extends another adds clauses to it and cites its clauses whole; GSG extends GKK.

GSG: Grassroots Social Graph [17]

GSG extends GKK.
Alice *trusts* Bob when she knows Bob personally and believes him to keep the undertakings he signs under grassroots contracts.
Alice and Bob are persons.

1. ◆ *Friendship.* Alice and Bob, who trust each other and are not friends, may jointly decide to become friends.
2. ■ *Ending a friendship.* Alice may end her friendship with Bob.
3. ○ *Loss of Trust.* Alice ends her friendship with Bob when she ceases to trust him.

GSG is the grassroots social graph [17, 39]. The GSG app is a mini-app of the grassroots app, compiled down the cascade from GSG; Figure 2 shows it realising GSG:1.

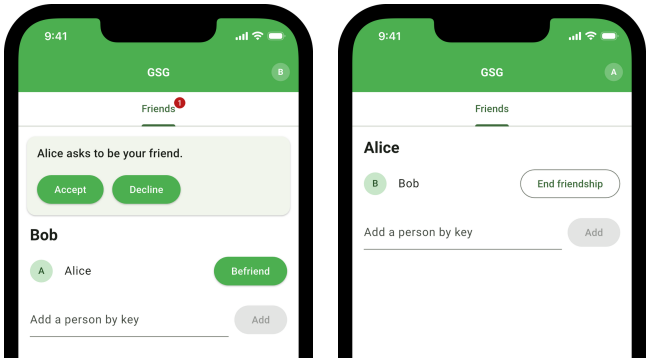


Figure 2: The GSG app: Alice asks to be Bob's friend, and Bob answers Accept or Decline (GSG:1); Alice's friends after Bob accepts.

2.3 The grassroots social network

GSN: Grassroots Social Network [17]

GSN amends GSG.
Alice and Bob are persons.

1. ■ *Messaging.* Alice may send a message to her friend Bob.
2. ■ *Group.* Alice may create a group, of which she is the leader and first member; Alice may close the group by leaving it.
3. ■ *Joining and Leaving.* Alice, the leader of a group, may add her friend Bob to it, with Bob's consent; either Alice or Bob may then end Bob's membership in the group, which also ends when Alice and Bob cease to be friends.
4. ■ *Group messaging.* A member of a group may send a message to the group, and its leader delivers it to every current member.

The GSN app is a mini-app of the grassroots app, compiled down the cascade from GSN; Figure 3 shows it realising GSN:2, GSN:3 and GSN:4.

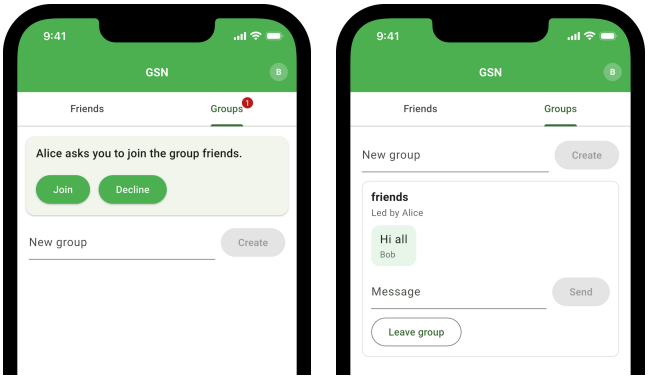


Figure 3: The GSN app, on Bob's phone: Alice asks Bob to join her group *friends*, and Bob answers Join or Decline (GSN:3); the group *friends*, led by Alice (GSN:2), with Bob's message to the group as Alice delivered it to him (GSN:4).

3 A SOCIAL GRAPH WITH ACCOUNTABLE AI AGENTS

This section presents a grassroots contract [6, 17] for a social graph in which AI agents take part, each with a principal and a developer accountable for it, and shows which parts of the second risk of Section 1 the contract excludes and which it makes answerable. The contract has three parts: the grassroots social graph with GKK (Section 2.2) and GnW, guardians and wards, both shared with the child-safe social network [43], whose third part is for parents and children; and the AI-agents extension, the third part here; Section 4 adds the social network of Section 2.3 and its provenance.

The running example is Alice's AI agent Alf befriending Bob, Acme Ltd being Alf's developer. It rests on three clauses: ■ Alf and Bob, who know each other, may become friends by an act of Alf, Alice and Bob, provided Alice and Bob are friends (GnW:3); ■ every act of Alf carries Alice's attestation, signed over Alf's public key, that Alf runs code signed by its developer, Acme Ltd (GAI:2); ○ Acme Ltd is accountable for Alf's defects, and Alice for operating Alf recklessly (GAI:1).

3.1 A Social Contract for Guardians and Wards

GnW is a contract for guardians and their wards [17]; GAI instantiates it for AI agents, and the child-safe social network for children [43].

GnW: Guardians 'n' Wards [17]

GnW amends GSG.
A *relation* is a friendship or the membership of a group. Bob trusts Carol, a ward of Alice, when he trusts Alice.
Alice, Bob and Erin are persons, Carol and Dave wards.

1. ■ *Creation.* Alice may create a *ward*, Carol, and establish Carol's identity by choosing Carol's key pair. Alice is then the *creator*, a *guardian* and a friend of Carol, a friendship Carol may not end.
2. ■ *Joint guardianship.* Bob, a friend of a guardian of Carol, may become a guardian and a friend of Carol by an act of Bob and the guardians of Carol; Bob's guardianship ends by an act of Bob, by an act of the other guardians of Carol, or when Bob ends his friendship with Carol. Carol may not end a friendship with her guardian.
3. ■ *Ward and person.* Carol, a ward of Alice, and Bob may come to know each other by an act of both. Carol enters a relation with Bob provided they know each other and Alice and Bob are friends, by an act of Carol, Alice and Bob. Carol, and the agent in a relation with Carol, may end it, as may Alice.
4. ■ *Ward and ward.* Carol, a ward of Alice, and Dave, a ward of Bob, may come to know each other by an act of both. Carol enters a relation with Dave provided they know each other, Alice knows Dave and is a friend of Bob, by an act of Carol, Alice, Dave and Bob, Alice and Bob possibly one person.
5. ■ *Retiring.* The guardians of Carol may retire Carol; every guardianship and relation of Carol then ends and Carol takes no act.

A ward may also be a business, whose guardians are its officers; we defer specifying how officers jointly exercise control over a business to future work.

An AI agent is made by a person, its *developer*, and is operated by its guardians, among them the person who created it, its *principal*. Accountability for an AI agent is divided between the two as it is for a car between its manufacturer and its driver: the manufacturer is accountable for manufacturing defects, and the driver for reckless driving. So the developer signs the code of the AI agent, and its principal attests, in a speech act signed over the AI agent's public key, that the AI agent runs that code, and every act of the AI agent carries the attestation.

GAI: Accountable AI Agents

GAI instantiates GnW, composes it with GFP [17] and restricts the composition.
Alice is a person and Alf an AI agent.

1. ○ *AI developer and guardian accountability.* The *developer* of an AI agent, Alf, signs Alf's code, and is accountable for Alf's defects. Every guardian of Alf is accountable for operating it recklessly. The creator of Alf is its *principal*.
2. ■ *AI acts.* Every act of Alf names its developer and its principal, and carries its principal's attestation, signed over Alf's public key, that Alf runs code signed by its developer.

Pragmas

AI agents: the AI agents Alice is a guardian of.
Creation: `Alice [Enter] <name>`
Carol: `Alice asks to be your principal.`
Joint guardianship: `Alice [Make guardian]`
Bob: `Alice asks you to be a guardian of her AI agent Carol.`
Erin: `Alice asks that Bob be a guardian of your AI agent Carol.`
`Alice [End guardian] <person>`
Erin: `Alice asks to end Bob's guardianship of your AI agent Carol.`
`Bob [Resign] [Guardian unfriend]`

Ward and person: `Bob [Agent acquaint] <person>`
Carol: `Bob wishes that you know each other.`
`Alice [Agent befriend]`
Carol: `Alice asks that you and Bob befriend.`
Bob: `Alice asks that you and her AI agent Carol befriend.`
`Alice [Agent unfriend] <person>`
`Carol [Agent end]`
Ward and ward: `Carol [Agents acquaint] <person>`
Dave: `Carol wishes that you know each other.`
`Alice [Agents befriend]`
Carol: `Alice asks that you and Dave befriend.`
Dave: `Alice asks that you and Carol befriend.`
Bob: `Alice asks that Carol and your AI agent Dave befriend.`
Retiring: `Alice [Retire]`
Erin: `Alice asks to retire your AI agent Carol.`

In GAI the wards are AI agents and their guardians the persons who operate them. The principal's will is given once, when they create the AI agent; after that the AI agent acts on its own, and a guardian takes part in each friendship it enters (GnW:3 and GnW:4): an AI agent comes to know an agent by an act of the two, and befriends one only with a guardian. The principal signs the attestation once too, when they create the AI agent, and the AI agent presents it to each agent it comes to know, who records it (GAI:2). The guardians of an AI agent may make a friend of one of them a guardian of it (GnW:2), and may retire it (GnW:5). A guardian signs the undertaking to keep the contract for their ward [17].

Figure 4 shows the contract on a person's phone, in the GSNP app of Section 4: a person creates an AI agent, attesting that it runs code signed by its developer, approves each friendship of the AI agent, and may retire it; every agent an AI agent knows sees its principal and its developer.

3.2 The speech-act schemas

The clauses of the contract are written as speech-act schemas [17], listed in Appendix B, which compile into volition-guarded multiagent atomic transactions [26], each a change to the states of its participants that occurs only if the agents guarding it are willing; the protocol realising the contract is the protocol over them [17, 26], recalled in Appendix A. A schema is written with its head before the arrow and its body after. The first argument of an atom is its holder, the agent in whose state it is; the body is what the act requires of the machines and, negated, forbids; the head is what it adds and, negated, deletes; and the guard names the agents who must will it. Every role requires *identity*, which is not shown. An agent p is a *person* when it holds $person(p)$, and an agent r is an *AI agent* when it holds $agent(r)$; its *principal* is the p with $principal(r, p)$, its *guardians* the p with $operated_by(r, p)$, each holding $operator_of(p, r)$, and its *attestation* the y with $attestation(r, y)$; the predicates are listed in Table 1 (Appendix B).

Example 3.1 (An AI agent coming to know a person, and befriending them). The two schemas below realise GnW:3 for an AI agent Carol, a guardian Alice of Carol and a person Bob: Carol, who has a guardian, and Bob come to know each other by an act of the two, Bob recording Carol with its attestation y (GAI:2), and then, Alice and Bob being friends, may become friends by an act of the three; a guardian of Carol, and Carol, may end the friendship by the third and fourth schemas of GnW:3, and Bob by that of GSG:2 (Appendix B).

GnW:3: Speech-Act Schemas

3. $knows(Carol, Bob)$, $knows(Bob, Carol)$,
$agent_of(Bob, Carol, y) \leftarrow$
$agent(Carol)$, $operated_by(Carol, Alice)$,
$attestation(Carol, y)$, $person(Bob)$ guarded by $Carol$, Bob.

$friend(Carol, Bob)$, $friend(Bob, Carol) \leftarrow$
$agent(Carol)$, $operated_by(Carol, Alice)$,
$knows(Carol, Bob)$, $\neg friend(Carol, Bob)$,
$knows(Bob, Carol)$, $person(Bob)$, $\neg friend(Bob, Carol)$,
$person(Alice)$, $operator_of(Alice, Carol)$,
$friend(Alice, Bob)$ guarded by $Carol$, Bob, $Alice$.

For example, let Alice and Bob be persons and Alf Alice's AI agent. Alice and Bob establish their identities, come to know each other and befriend; Alice signs y, her attestation that Alf, by its public key, runs the code its developer signed, and creates Alf with it; Alf and Bob come to know each other, and Bob records Alf with Alice's attestation; Alf befriends Bob, with Alice willing; Alf signs an item x and sends it to Bob; and Alice retires Alf, which ends Alf's friendship with Bob. The atoms each act adds and deletes are in Table 2 (Appendix B), Bob's record and the item remaining with him after the friendship ends.

3.3 Properties

This section states what the schemas enforce about an AI agent and its principal, with the argument for it, and that the protocol is volitionally grassroots, proved in Appendix C.

The standing of a principal. Consider any configuration reached by a run of the protocol (Appendix B) and an AI agent r in it, with $principal(r, p)$.

(1) *Creation.* The identity of r was established by an act of GnW:1 with p and r in its roles, guarded by both, for an item y that p holds and authored, and $agent(r)$, $principal(r, p)$ and $attestation(r, y)$ hold in every configuration since; r holds $operated_by(r, g)$ if and only if g holds $operator_of(g, r)$; and every guardian g of r is a person and holds $agent_of(g, r, y)$.
(2) *Whom r knows and befriends.* Every $s \neq p$ with $knows(r, s)$ became a guardian of r by an act of GnW:2 or came to know r by an act of GnW:3 or GnW:4, and holds $agent_of(s, r, y)$; every s with $friend(r, s)$ became a friend of r on becoming a guardian of r, by an act of GnW:1 or GnW:2, or is known to r and became a friend of r by an act of GnW:3 or GnW:4 guarded by a guardian of r.
(3) *The power to end each.* For every guardian g of r and every friend s of r that is not a guardian of r, the act of GnW:3 by which g ends the friendship of r and s is machine-enabled and guarded by g; and while r has a guardian, the act of GnW:5 by which the guardians of r retire it, ending every friendship

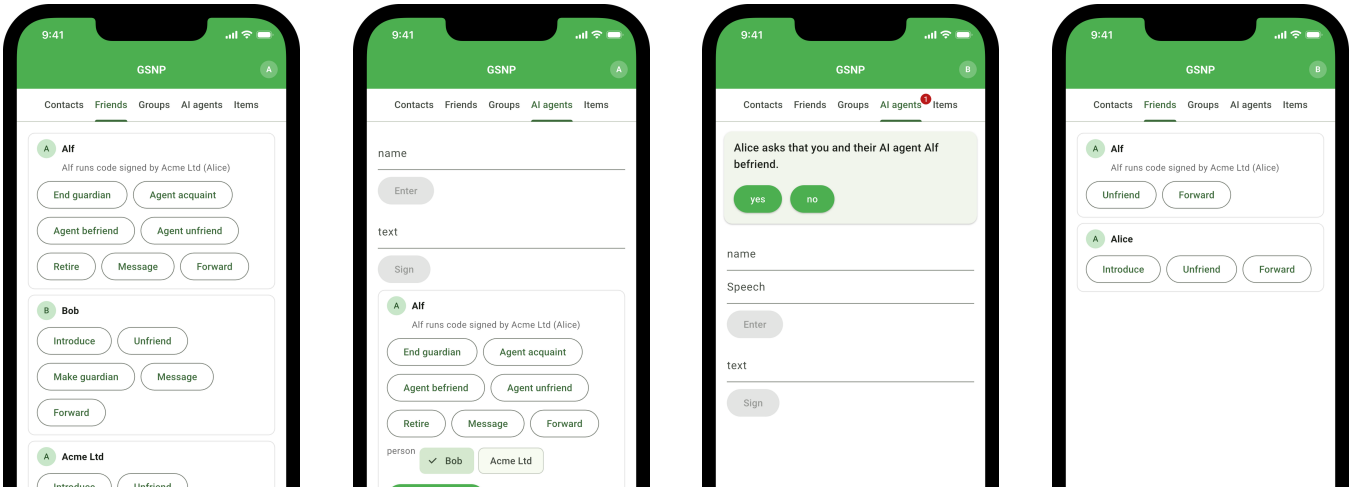


Figure 4: Accountable AI agents, in the GSNP app: Alice's AI agent Alf, its code signed by Acme Ltd and attested by her, and its Retire button (GnW:1, GAI:2 and GnW:5); Alice proposes that Alf befriend Bob (GnW:3); Bob is asked to befriend Alf, an AI agent of Alice (GAI:2); Bob's friends, Alf among them with its principal and its developer.

and guardianship of r, is machine-enabled and guarded by the guardians of r.

(4) *An AI agent with no guardian.* If r has no guardian then r has left, and $principal(r, p)$ and $attestation(r, y)$ hold; a retired AI agent has no guardian.

Each holds by inspection of the schemas. (1) Only the schema of GnW:1 adds *agent*, *principal* and *attestation*, and no schema deletes them, so $agent(r)$, $principal(r, p)$ and $attestation(r, y)$ were added by one act of it, which requires $item(p, y)$ with $author(y) = p$, forbids $identity(r)$ and is guarded by p and r; before it r held nothing, as every other schema requires *identity* at each of its roles. The schemas of GnW:1 and GnW:2 add $operated_by(r, g)$ at r together with $operator_of(g, r)$ and $agent_of(g, r, y)$ at g, where they require *person*, the second requiring $agent_of(\cdot, r, y)$ at a guardian of r; those of GnW:2 by which a guardianship ends and that of GnW:5 delete $operated_by(r, g)$ and $operator_of(g, r)$ together; and no schema deletes *person* or an atom of $agent_of$. (2) The schemas adding $knows(r, \cdot)$ are those of GKK:2 and GKK:3, which require *person* at their roles, and r, holding *agent*, is not a person, a schema adding *identity* adding *person* or *agent* and not both; that of GnW:1, which adds $knows(r, p)$; that of GnW:2, which adds $knows(r, g)$ for the new guardian g, with $agent_of(g, r, y)$ at g; and those of GnW:3 and GnW:4 for knowing, which require $attestation(r, y)$ and add $agent_of(\cdot, r, y)$ at the agent coming to know r. The schemas adding $friend(r, \cdot)$ are that of GSG:1, which requires *person* at its roles; those of GnW:1 and GnW:2, which add a guardian; and those of GnW:3 and GnW:4 for a relation, which require that r know the new friend and are guarded by a guardian of r. (3) The bodies of the two acts hold: r holds $operated_by(r, g)$ and not $operated_by(r, s)$; g is a person and holds $operator_of(g, r)$, and so does every guardian of r, by (1); and every friend of r holds *friend* of r, the schemas adding and deleting the two atoms of a friendship together. (4) Every role r guards in a schema other than that of GnW:1, which forbids *identity* there, requires *person*, which r does not hold, or an atom of *operated_by* at r, the schemas of GFP:1, GFP:2 and GSN:1 asking that their agent be a person or an AI agent holding one; and the one schema adding *operated_by* at r once it has an identity, that of GnW:2, requires one there. So once r holds no atom of *operated_by* no act r guards is machine-enabled: r has left; $principal(r, p)$ and $attestation(r, y)$ hold by (1); and the schema of GnW:5 clears *operated_by* at r.

Part (1) is why what the contract says of an AI agent is enforced rather than attested: an agent the contract records as an AI agent of p cannot cease to be one by any act, its own or another's, and by part (4) it takes no act without a guardian. Retirement ends every guardianship and friendship of the AI agent (GnW:5): the retired AI agent takes no act; the record of its principal stays, for its past acts.

Grassroots. The protocol realising the contract is volitionally grassroots (Proposition C.1, Appendix C, by the theorem for contracts with wards of [17]): two persons can always befriend, an AI agent with a guardian can always come to know a person or another AI agent with a guardian, and befriend them with the guardians willing, and through befriending any agent can come to interact with any other; two groups that have formed apart, each with an agent that has not left, can always coalesce, and only by an act an agent of each wills. An AI agent with no guardian has left.

3.4 What the contract excludes and what it makes answerable

A clause is enforced, attested or undertaken (Section 2) [17]: no act of agents can breach an enforced clause; a breach of an attested clause leaves non-repudiable evidence against the agent in breach in another agent's hands; and an undertaken clause is kept outside the contract, the agent's signed undertaking to keep the contract, held by every agent they interact with, being its record, so a breach of an attested or an undertaken clause can be taken to court. The second risk of Section 1 has five parts, and we take them in turn.

Anonymity. A person signs their acts with the private key of their genuine digital identity (GKK:1); an AI agent's key pair is chosen by its principal (GnW:1), and every act of an AI agent is signed with its private key and names its principal (GAI:2). Both are enforced:

an agent without an identity has no act of the contract available to it, and an act of the contract is a digital speech act, an utterance signed with the key of the agent taking it [17, 18].

Masquerading as a person. An AI agent is recorded as one on its creation (GnW:1 and GAI:2), is a ward from its first act and remains one (Section 3.3(1)), every agent it comes to know records it with its principal's attestation, and every friendship of it is an act of a guardian of it (GnW:3, GnW:4 and GAI:2), so every agent an AI agent knows, and so every agent it interacts with, holds a record of it with its principal's attestation, which names it as an AI agent and names its principal (Section 3.3(2)): enforced. What the contract cannot determine is whether an agent that chose its own key pair is a person: no act of the contract can tell a person's act from a program's taken under that person's identity. Such a program is not an AI agent of the contract; its acts are signed with the person's private key and are that person's acts, for which the person is accountable (GKK:1). A person's accountability for the acts signed with their private key is undertaken: the person's signed undertaking, held by every agent they interact with, is its record.

Unknown masters. The principal of an AI agent is a person with a genuine digital identity, recorded in the AI agent and named in each of its acts, and a guardian of it, a person, takes part in each of its friendships (GnW:3, GnW:4 and GAI:2): enforced.

Unknown goals and restraints. The contract does not determine what an AI agent is for or what its principal lets it do. It determines who is accountable for it: its developer for its defects, and every guardian of it for operating it recklessly (GAI:1). The clause is undertaken, each guardian's signed undertaking being held by every agent they interact with. GAI:2 is enforced: the principal signs the attestation when they create the AI agent, and every act of the AI agent carries it, every agent the AI agent knows holding it (Section 3.3(1) and (2)). The attestation does not establish what code the AI agent runs; an attestation of code the AI agent does not run is evidence against the principal that they cannot repudiate.

Accountability and culpability. Every act of an AI agent is signed with its key, names its principal and carries its principal's attestation (GAI:2), and the agent it is addressed to holds it: non-repudiable evidence attributing the act to the AI agent and the AI agent to its principal [18]. Harm done by an AI agent under this contract is harm done by an identified agent, whose guardians are accountable for operating it recklessly and whose developer is accountable for its defects (GAI:1).

Hence on a platform realising this contract there are no anonymous agents, natural or computational, and so there are no anonymous AI agents: an AI agent acts under an identity of its own, recorded as an AI agent and naming its principal, or a program acts under a person's identity and that person is accountable for its acts.

4 A SOCIAL NETWORK WITH DIGITAL PROVENANCE

This section presents GSNP, the grassroots social network with provenance [39], which composes the contract of Section 3 with GSN (Section 2.3) and amends the composition, so that a message carries an item of GFP, grassroots forwarding with provenance [17]: an agent may create an item by signing it, and forward an item they hold to a friend, adding their signature to the signatures the item carries. It then shows which parts of the third risk of Section 1 the contract excludes and which it makes answerable. An *item* is a speech act: content signed by its author [18], and, when forwarded, signed over by each agent that forwarded it, in order. GSNP's speech-act schemas are in the repository, and the protocol realising it is volitionally grassroots (Proposition C.2, Appendix C).

4.1 Extending the contract with provenance

GFP: Grassroots Forwarding with Provenance [17]

GFP extends GSG.
The author of an item is its first signer.
Alice and Bob are persons.

1. ■ *Signing.* Alice may create an item by signing it.
2. ■ *Forwarding.* Alice may forward an item she holds to her friend Bob, adding her signature to the signatures the item carries; Bob sees them.

GSNP: Grassroots Social Network with Provenance

GSNP composes GAI with GSN [17] and amends the composition.
Alice and Bob are agents.

Pragmas

Items: the items Alice holds.
Messaging: `Alice [Message]`
Group messaging: `Bob [Post]`
`Alice [Post own]`
Forwarding: `Alice [Forward]`
`Bob [Post forward]`

GSNP adds provenance to the messages of GSN: a message to a friend (GSN:1) or to a group (GSN:4) carries an item. An agent signs the content they create, as they sign every act (GKK:1 and GFP:1): signing adds the item to its author's holdings; a message carries an item its sender authored, as they signed it; forwarding adds to the friend's holdings an item the forwarder holds, signed over by the forwarder (GFP:2); and a leader delivers a member's item to the group by forwarding it, signed over by the leader; so an item in an agent's hands is signed by its author and by every agent that forwarded it, in order, and carries its provenance inside it. Alice may remove the signatures of an item with other software and send the result as her own; it is then an item Alice created, of which she is the author (GFP:1), and she is accountable for it. An agent holds an item by signing it or by receiving it from a friend, the leader of a group being a friend of each of its members (GSN:3), and in no other way: content that reaches a person otherwise is not an item of the platform. Accountability for the items of an AI agent is as in GAI:1.

The GSNP app is the GSN app with provenance and with the AI agents of Section 3 (Figure 5): every item shows its author and every agent that forwarded it (GFP:2).

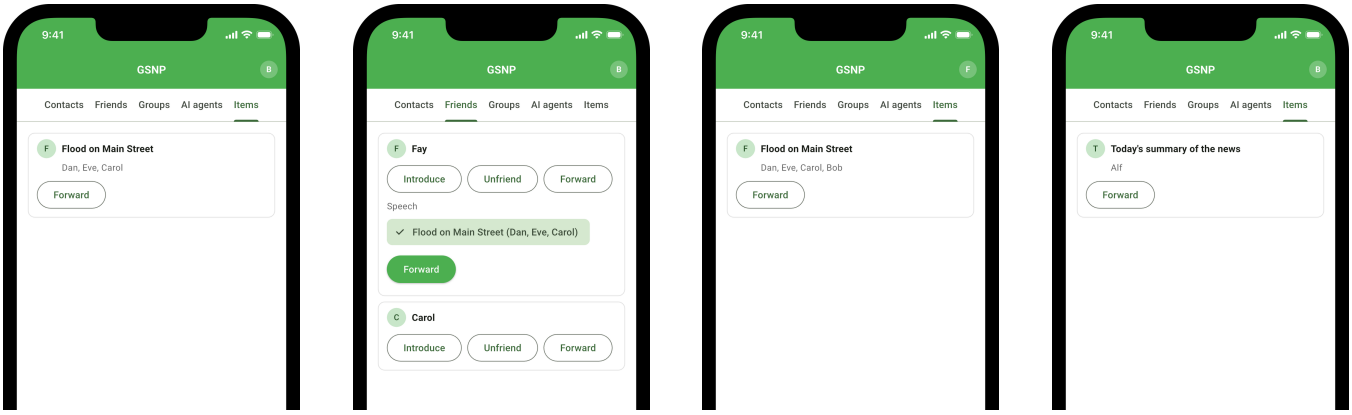


Figure 5: The GSNP app: on Bob's phone, an item with its author and the agents that forwarded it (GFP:2); Bob forwarding it to Fay (GFP:2); on Fay's phone, the item with Bob among its forwarders; on Bob's phone, an item authored by the AI agent Alf (GAI:2).

4.2 What the contract excludes and what it makes answerable

The third risk of Section 1 has three parts, and we take them in turn, with the three modalities of Section 3.4.

Anonymous content. Every item is signed by its author, an agent with a genuine digital identity who signs every act with their private key (GKK:1), and an agent holds an item only by signing it or by receiving it from a friend, who authored it or signed it over (GFP:2): enforced. Every item an agent holds carries the signatures of its author and of each agent it passed through, and content signed by nobody is not an item of the platform. Whether the content was produced by a person or by an AI is not what the signatures say; who is accountable for it is.

Content people cannot tell from genuine content. The platform does not decide whether content is synthetic; it decides who signed it. An item authored by an AI agent is signed by the AI agent, which every agent it knows holds recorded as an AI agent of its principal (GnW:3 and GAI:2): a reader who knows the AI agent sees that the item is an AI agent's and whose; enforced. A reader who does not know the AI agent sees its signature and the forwarders', each of whom received it from a friend (GFP:2). An item a person signs is that person's, whatever produced it, and the person is accountable for it (GKK:1): a person who signs AI-generated content as their own is its author. An item that purports to be a person's and is not carries that person's signature or it does not, and a signature cannot be forged [18]; an item that depicts or quotes a person is its author's statement about them, and the author is accountable for it. So a reader of a grassroots social network tells content with provenance from the rest by its signatures, which every item carries, and not by its look.

Producers evading accountability. The signature of an item is non-repudiable evidence that its author produced it, held by every agent that holds the item, and each forwarder's signature over it is evidence that they passed it on: enforced (GFP:2). An agent who removes the signatures of an item they received and sends the result as their own is its author (GFP:1), and is accountable for it. The author of an item, a person or an AI agent with an accountable developer and principal (GAI:1), and each agent that forwarded it, are identified agents, and harm done by an item can be taken to court against them [17, 18].

Hence on a platform realising this contract there is no anonymous content: every item carries the signatures of its author, a person accountable for it or an AI agent whose principal and developer are accountable for it, and of every agent that forwarded it. Deepfake content may be produced outside the platform and signed into it, and it then has an author who is accountable for it; what the platform does not carry is content with no one behind it.

5 RELATED WORK

AI risks. Taxonomies and surveys of AI risks include those of Weidinger et al. [56], Bommasani et al. [3] and Hendrycks et al. [20], the AI Risk Repository [52] and the International AI Safety Report [2]. Algorithmic governance and regulation [10, 58] and gradual disempowerment [24] bear on the first risk; personhood credentials [1], social bots [15], visibility into AI agents [7], the governance of agentic systems [51], moral crumple zones [13] and meaningful human control [34] on the second; deepfakes [9, 28, 30, 55] and reducing the risks of synthetic content [8] on the third; and AI assistants [16], chatbot use [14] and malicious use [5, 27] on the two risks not addressed here. We address the three through the organisation of the platform.

Grassroots platforms. Grassroots platforms [38, 40] are characterised against global platforms, centralised, decentralised and federated [41], the decentralised ones exemplified by Bitcoin [29] and Ethereum [57]. They are specified by atomic transactions [45] and volition-guarded multiagent atomic transactions [26], and defined by grassroots contracts [17], digital social contracts [6] meeting Proudhon's conditions [33]. Their instances include grassroots social networks [39], currencies [37] and bonds [44], and a child-safe social network [43]. Their identities are genuine personal identifiers [35], their communities grow sybil-resiliently [32], their content is digital speech acts [18], and their participants recover from the loss or compromise of a key or a device with the help of their friends [12].

The cascade. A contract is compiled down to a phone through volition elicitation and vGLP [22], Grassroots Logic Programs [42],

their types [49] and their implementation [46], and the Grassroots Super-App [11], in which each platform is a mini-app.

Person-to-person systems. Two deployed systems share the person-to-person architecture of grassroots platforms. In Secure Scuttlebutt [54] each participant publishes to a single-writer, append-only log and replicates the logs of the identities they choose, so data flows along the edges of a social graph, as in grassroots social networks [39]. Briar [4] synchronises messages directly between its users' devices, with no central server, and over Bluetooth or Wi-Fi when the Internet is down. Both are classified as grassroots by their essential agents, informally [41]. Local-first software [23] keeps the primary copy of a person's data on their own devices, with servers holding secondary copies, and merges concurrent changes by conflict-free replicated data types that sync over any channel, a peer-to-peer connection included; in a grassroots platform, the copies of a person's state are held by the state custodians they designate among their friends [12].

6 CONCLUSION

Grassroots platforms corral three AI risks of digital platforms. A grassroots platform has no part whose capture is control of it, so to control it an AI must control the smartphones of its participants, one by one. Under the contract of Section 3, every AI agent has a principal and a developer, every act of an AI agent names its principal and carries its principal's attestation that it runs code signed by its developer, and its guardians and its developer are accountable for it. Under the contract of Section 4, every item carries the signatures of its author and of every agent that forwarded it.

REFERENCES

[1] Steven Adler et al. 2024. Personhood credentials: Artificial intelligence and the value of privacy-preserving tools to distinguish who is real online. https://doi.org/10.48550/arXiv.2408.07892 arXiv:2408.07892

[2] Y. Bengio et al. 2026. *International AI Safety Report 2026.* DSIT 2026/001. Department for Science, Innovation and Technology. https://internationalaisafetyreport.org/publication/international-ai-safety-report-2026

[3] Rishi Bommasani et al. 2021. On the Opportunities and Risks of Foundation Models. arXiv:2108.07258. https://doi.org/10.48550/arXiv.2108.07258 arXiv:2108.07258

[4] Briar Project. 2018. Briar: Secure messaging, anywhere. https://briarproject.org.

[5] Miles Brundage et al. 2018. The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation. https://doi.org/10.48550/arXiv.1802.07228 arXiv:1802.07228

[6] Luca Cardelli et al. 2020. Digital social contracts: A foundation for an egalitarian and just digital society. In *CEUR Proceedings of the First International Forum on Digital and Democracy*, Vol. 2781. CEUR-WS, 51–60. https://ceur-ws.org/Vol-2781/paper5.pdf

[7] Alan Chan et al. 2024. Visibility into AI Agents. In *The 2024 ACM Conference on Fairness, Accountability, and Transparency*. Association for Computing Machinery, 958–973. https://doi.org/10.1145/3630106.3658948

[8] Bilva Chandra et al. 2024. *Reducing Risks Posed by Synthetic Content: An Overview of Technical Approaches to Digital Content Transparency.* NIST AI 100-4. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.100-4

[9] Robert Chesney and Danielle Keats Citron. 2019. Deep fakes: a looming challenge for privacy, democracy, and national security. *California Law Review* 107 (2019), 1753.

[10] John Danaher et al. 2017. Algorithmic governance: Developing a research agenda through the power of collective intelligence. *Big Data & Society* 4, 2 (2017). https://doi.org/10.1177/2053951717726554

[11] Ohad Eitan, Idit Keidar, and Ehud Shapiro. 2026. The Grassroots Super-App: A Secure Operating System for Grassroots Mini-Apps. arXiv:XXX:XXXXX. arXiv:XXX:XXXXX [cs.DC]

[12] Ohad Eitan, Idit Keidar, and Ehud Shapiro. 2026. Securing People and their Machines Against Major Faults (Full Version). *Proceedings of The 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026), to appear* (2026). https://arxiv.org/abs/2607.02304 arXiv:2607.02304.

[13] Madeleine Clare Elish. 2019. Moral Crumple Zones: Cautionary Tales in Human-Robot Interaction. *Engaging Science, Technology, and Society* 5 (2019), 40–60. https://doi.org/10.17351/ests2019.260

[14] Cathy Mengying Fang et al. 2025. How AI and Human Behaviors Shape Psychosocial Effects of Extended Chatbot Use: A Longitudinal Randomized Controlled Study. https://doi.org/10.48550/arXiv.2503.17473 arXiv:2503.17473

[15] Emilio Ferrara et al. 2016. The Rise of Social Bots. *Commun. ACM* 59, 7 (2016), 96–104. https://doi.org/10.1145/2818717

[16] Iason Gabriel et al. 2024. The Ethics of Advanced AI Assistants. arXiv:2404.16244. https://doi.org/10.48550/arXiv.2404.16244 arXiv:2404.16244

[17] James Golike, Andy Lewis-Pye, and Ehud Shapiro. 2026. Grassroots Social Contracts: From Legal Text to an App (Full Version). *arXiv:2609.06231* (2026).

[18] James Golike and Ehud Shapiro. 2026. Digital Speech Acts Retain Control of Copyright with People, Not Platforms. *arXiv preprint arXiv:2606.19263* (2026). https://arxiv.org/abs/2606.19263

[19] Daniel Halpern, Ariel D Procaccia, Ehud Shapiro, and Nimrod Talmon. 2024. Federated Assemblies. *Proc AAAI 2025; arXiv preprint arXiv:2405.19129* (2024). https://arxiv.org/abs/2405.19129

[20] Dan Hendrycks, Mantas Mazeika, and Thomas Woodside. 2023. An Overview of Catastrophic AI Risks. arXiv:2306.12001. https://doi.org/10.48550/arXiv.2306.12001 arXiv:2306.12001

[21] Idit Keidar, Andrew Lewis-Pye, and Ehud Shapiro. 2025. Constitutional Consensus. *arXiv preprint arXiv:2505.19216* (2025). https://arxiv.org/abs/2505.19216

[22] Idit Keidar and Ehud Shapiro. 2026. Volition Elicitation: Operational Semantics for People and Their Machines. arXiv:2607.14138. arXiv:2607.14138 [cs.DC] https://arxiv.org/abs/2607.14138

[23] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-First Software: You Own Your Data, in spite of the Cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '19)*. 154–178. https://doi.org/10.1145/3359591.3359737

[24] Jan Kulveit et al. 2025. Gradual Disempowerment: Systemic Existential Risks from Incremental AI Development. arXiv:2501.16946. https://doi.org/10.48550/arXiv.2501.16946 arXiv:2501.16946

[25] Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. 2023. Grassroots Flash: A Payment System for Grassroots Cryptocurrencies. *arXiv preprint arXiv:2309.13191* (2023). https://arxiv.org/abs/2309.13191

[26] Andy Lewis-Pye and Ehud Shapiro. 2026. Volition-Guarded Multiagent Atomic Transactions: Describing People and their Machines. *arXiv preprint arXiv:2604.25596* (2026). https://arxiv.org/abs/2604.25596

[27] Nahema Marchal et al. 2024. Generative AI Misuse: A Taxonomy of Tactics and Insights from Real-World Data. arXiv:2406.13843. https://doi.org/10.48550/arXiv.2406.13843 arXiv:2406.13843

[28] Yisroel Mirsky and Wenke Lee. 2021. The Creation and Detection of Deepfakes: A Survey. *Comput. Surveys* 54, 1 (2021), 1–41. https://doi.org/10.1145/3425780

[29] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf

[30] Sophie J. Nightingale and Hany Farid. 2022. AI-synthesized faces are indistinguishable from real faces and more trustworthy. *Proceedings of the National Academy of Sciences* 119, 8 (2022), e2120481119. https://doi.org/10.1073/pnas.2120481119

[31] Joon Sung Park et al. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*. 1–22. https://doi.org/10.1145/3586183.3606763

[32] Ouri Poupko, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. 2021. Building a sybil-resilient digital community utilizing trust-graph connectivity. *IEEE/ACM transactions on networking* 29, 5 (2021), 2215–2227. https://doi.org/10.1109/tnet.2021.3084303

[33] Pierre-Joseph Proudhon and John Beverley Robinson. 2004. *General idea of the revolution in the nineteenth century.* Courier Corporation. Original work published 1851.

[34] Filippo Santoni de Sio and Jeroen van den Hoven. 2018. Meaningful Human Control over Autonomous Systems: A Philosophical Account. *Frontiers in Robotics and AI* 5 (2018), 15. https://doi.org/10.3389/frobt.2018.00015

[35] Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. 2020. Genuine Personal Identifiers and Mutual Sureties for Sybil-Resilient Community Growth. In *International Conference on Social Informatics*. Springer, Springer, EU, 320–332. https://doi.org/10.1007/978-3-030-60975-7_24

[36] Ehud Shapiro. 2021. Multiagent Transition Systems for Composing Fault-Resilient Protocol Stacks. *arXiv preprint arXiv:2112.13650* (2021). https://arxiv.org/abs/2112.13650

[37] Ehud Shapiro. 2022. Grassroots Currencies: Turning Mutual Trust into Liquidity. *arXiv preprint arXiv:2202.05619* (2022). https://arxiv.org/abs/2202.05619

[38] Ehud Shapiro. 2023. Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391).* LIPICS, Italy, 47:1–47:7. https://arxiv.org/abs/2301.04391

[39] Ehud Shapiro. 2023. Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp. In *OASIS '23* (Rome, Italy). Association for Computing Machinery. https://doi.org/10.1145/3599696.3612898

[40] Ehud Shapiro. 2024. A Grassroots Architecture to Supplant Global Digital Platforms by a Global Digital Democracy. *arXiv:2404.13468, Proceedings of DAWO'24* (2024). https://arxiv.org/abs/2404.13468

[41] Ehud Shapiro. 2025. Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots. *arXiv preprint arXiv:2511.03286* (2025). https://arxiv.org/abs/2511.03286

[42] Ehud Shapiro. 2025. GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133* (2025). https://arxiv.org/abs/2510.15747

[43] Ehud Shapiro. 2026. Child-Safe Social Networking: From a Grassroots Social Contract to an App. *In preparation* (2026).

[44] Ehud Shapiro. 2026. Grassroots Bonds: Financing by the People, for the People. arXiv:2603.13671. arXiv:2603.13671 https://arxiv.org/abs/2603.13671

[45] Ehud Shapiro. 2026. Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations. In *Proceedings of the 27th International Conference on Distributed Computing and Networking*. 71–81. https://doi.org/10.1145/3772290.3772309 arXiv preprint arXiv:2502.11299.

[46] Ehud Shapiro. 2026. Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI (Full Version). *arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26* (2026). https://arxiv.org/abs/2602.06934

[47] Ehud Shapiro. 2026. sGLP: Simulating People with Stochastic Agents. arXiv:2610.14738. arXiv:2610.14738 [cs.MA]

[48] Ehud Shapiro. 2026. Sovereign Grassroots Currencies: A CBDC Architecture for Credit and Monetary Policy. arXiv:2609.27727. arXiv:2609.27727 https://arxiv.org/abs/2609.27727

[49] Ehud Shapiro. 2026. Types for Grassroots Logic Programs (Full version). *arXiv preprint arXiv:2601.17957* (2026). https://arxiv.org/abs/2601.17957

[50] Ehud Shapiro and Nimrod Talmon. 2026. Constitutional Governance in Metric Spaces. *Proceedings of the International Conference on Algorithmic Decision Theory (ADT 2026), to appear* (2026). https://arxiv.org/abs/2605.13362 arXiv:2605.13362.

[51] Yonadav Shavit et al. 2023. *Practices for Governing Agentic AI Systems*. White paper. OpenAI. https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf

[52] Peter Slattery et al. 2026. The AI risk repository: A meta-review, database, and taxonomy of risks from artificial intelligence. *Patterns* (2026), 101517. https://doi.org/10.1016/j.patter.2026.101517 arXiv:2408.12622

[53] Nimrod Talmon and Ehud Shapiro. 2025. Grassroots Federation: Fair Democratic Governance at Scale. *arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26* (2025). https://arxiv.org/abs/2505.02208

[54] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. 2019. Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM conference on information-centric networking*. 1–11. https://doi.org/10.1145/3357150.3357396

[55] Cristian Vaccari and Andrew Chadwick. 2020. Deepfakes and Disinformation: Exploring the Impact of Synthetic Political Video on Deception, Uncertainty, and Trust in News. *Social Media + Society* 6, 1 (2020). https://doi.org/10.1177/2056305120903408

[56] Laura Weidinger et al. 2022. Taxonomy of Risks Posed by Language Models. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. Association for Computing Machinery, 214–229. https://doi.org/10.1145/3531146.3533088

[57] Gavin Wood et al. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 151, 2014 (2014), 1–32. https://ethereum.github.io/yellowpaper/paper.pdf

[58] Karen Yeung. 2018. Algorithmic regulation: A critical interrogation. *Regulation & Governance* 12, 4 (2018), 505–523. https://doi.org/10.1111/rego.12158

[59] Shoshana Zuboff. 2019. *The age of surveillance capitalism: The fight for a human future at the new frontier of power.* Public Affairs, US. https://www.hachettebookgroup.com/titles/shoshana-zuboff/the-age-of-surveillance-capitalism/9781610395694/

A THE FRAMEWORK, RECALLED

This appendix recalls, in the words of [26], the mathematical foundations on which the paper is based; the proofs of the results recalled are in that paper.

Each agent consists of a *person* and a *machine* (e.g., smartphone) operated by the person. A *machine transaction* specifies what machines do; a *volition guard* specifies which persons must be willing for the transaction to occur.

We assume a potentially infinite set of *agents* Π, but consider only finite subsets of it, so we assume any particular set of agents $P \subset \Pi$ to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible. As standard, we use S^P to denote the set of all total functions from P to S, and if $c \in S^P$ we use c_p to denote the value of c at $p \in P$.

Definition A.1 (Machine State, Configuration, Transaction, Volition–Guarded Multiagent Atomic Transaction [26]). Given an arbitrary set S of *machine states*, with a designated *initial state* $s_0 \in S$, and agents $Q \subset \Pi$, a *machine configuration* over Q is a member of S^Q, and a *machine transaction* over *participants* Q is a pair $c \to c' \in (S^Q)^2$ such that $c \neq c'$; it is *unary* if $|Q| = 1$. Given such a machine transaction t, a *volition-guarded multiagent atomic transaction* over t—henceforth, *volition-guarded transaction*—is a pair (t, Q') where $Q' \subseteq Q$ are its *guards*.

A volition-guarded transaction can be carried out only if its guards are willing; its participants include those whose state changes and those whose state is a precondition and does not change.

Distinct machine transactions can represent "the same action" in different configurations; this is captured with an equivalence relation on machine transactions.

Definition A.2 (Transaction Equivalence [26]). Given a set of machine transactions R, a *transaction equivalence* is an equivalence relation $\sim$ on R such that $t \sim t'$ implies t and t' have the same participants, and $R/\sim$ is countable. We write $[t]$ for the equivalence class of t under $\sim$.

For example, all befriend(p, q) transactions—differing only in the configurations in which they occur—form an equivalence class. Next is the mathematical machinery allowing people to express their volitions regarding the classes of machine transactions their machines may participate in.

Definition A.3 (Agent State and Configuration [26]). Given agents P, states S with initial state s_0, a set of machine transactions T each over its own participants $Q \subseteq P$ and S, and equivalence $\sim$ on T, an *agent state* is a pair $(V, m) \in \mathcal{A} = (2^{T/\sim} \times S)$ where V is its *volitional state* and $m \in S$ its *machine state*. The *initial agent state* is $(\emptyset, s_0)$. An *agent configuration* c over P, S, T, and $\sim$ is a member $c \in \mathcal{A}^P$ in which $c^v_p \subseteq (T/\sim)_p$ for every $p \in P$, where $(T/\sim)_p$ denotes the classes in $T/\sim$ in which p is a participant; we write c^v_p for the volitional state and c^m_p for the machine state of agent p in c.

Definition A.4 (Volitional Transaction [26]). Given agents P, states S, machine transactions T over P and S, and equivalence $\sim$ on T:

(1) A *volition change of agent* $p \in P$ is a pair $c \to c'$ of agent configurations over $\{p\}$, S, T, and $\sim$ such that $c^v_p, {c'}^v_p \subseteq (T/\sim)_p$ and $c^v_p \neq {c'}^v_p$, and $c^m_p = {c'}^m_p$.
(2) A *volitional machine transaction* induced by a volition-guarded transaction (t, Q'), for some $t = (d \to d') \in T$ over $Q' \subseteq Q \subseteq P$, is a pair $c \to c'$ where $c \neq c'$ are agent configurations over P, S, T, and $\sim$ such that $[t] \in c^v_q$ for every $q \in Q'$; $c^m_p = d_p$ and ${c'}^m_p = d'_p$ for every $p \in Q$; $c^m_p = {c'}^m_p$ for every $p \in P \setminus Q$; and ${c'}^v_p = c^v_p \setminus \{[t]\}$ for every $p \in P$.
(3) A *volitional transaction* is a volition change or a volitional machine transaction.

When a volitional machine transaction induced by (t, Q') is taken, the class $[t]$ is removed from every agent's volitional state: the will is fulfilled by any equivalent transaction. A person may independently change their volitional state via volition changes, which may add or remove classes; beyond these, the framework removes a class from c_p^v only upon fulfilment.

A person's not willing a transaction has, so far, no consequence: no volitional transaction is enabled by the absence of a will. An implementation of volition-guarded transactions by communicating agents [22] needs one, so that a machine asked for its person's will answers either way; it is the following, by which the machine answers for a person who does not will an act: in machine state s, if p wills it s' is taken, else s''.

Definition A.5 (Volition-Guarded If-Then-Else Transaction, Then-Branch, Else-Branch [26]). Given machine states S, a *volition-guarded if-then-else transaction* over a single participant $p \in \Pi$ is a triple $s \to s'$; s'' *guarded by* p, with $s, s', s'' \in S$, $s \neq s'$ and $s \neq s''$; the unary machine transaction $s \to s'$ is its *then-branch* and $s \to s''$ its *else-branch*.

Definition A.6 (Else Transaction [26]). Given agents P, states S, machine transactions T over P and S, and equivalence $\sim$ on T, an *else transaction* induced by a volition-guarded if-then-else transaction $s \to s'$; s'' guarded by $p \in P$, with $s \to s' \in T$, is a pair $c \to c'$ of agent configurations over P, S, T, and $\sim$ such that $[s \to s'] \notin c_p^v$; ${c_p}^m = s$ and ${c'}_p^m = s''$; ${c_q}^m = {c'}_q^m$ for every $q \in P \setminus \{p\}$; and ${c'}_q^v = c_q^v$ for every $q \in P$.

The then-branch, taken when p wills it, is the unary volition-guarded transaction $(s \to s', \{p\})$ and induces volitional machine transactions per Definition A.4; a unary volition-guarded transaction is thus the special case with no else-branch. The else transaction is taken when p does not will the then-branch; it fulfils no will and removes no class. The two are placed in one class below, so that liveness obliges the machine to answer either way; the obligation is the machine's, and it has no time bound. Only a unary transaction has an else-branch: a binary one would let an agent that never wills it trigger it.

Induced transition systems. The following is a simplified variation, sufficient for the purpose of this work, on the foundations introduced in [36].

Definition A.7 (Transition System, Computation, Run, Safe, Live, Correct [26]). A *transition system* is a tuple $TS = (S, s_0, T, \sim)$, where:

(1) S is an arbitrary non-empty set, referred to as the set of *states*.
(2) Some $s_0 \in S$ is the designated *initial state*.
(3) $T \subseteq S^2$ is a set of *correct transitions over* S, where each transition $t \in T$ is a pair (s, s') of non-identical states $s \neq s' \in S$, also written as $t = s \to s'$.
(4) $\sim$ is a *partial equivalence relation* on T: a symmetric and transitive relation on T, not necessarily reflexive. Its domain $\{t \in T : t \sim t\}$ is partitioned into countably many *liveness classes* $T/\sim$; a transition outside the domain belongs to no class.

A *computation* of TS is a (nonempty, potentially infinite) sequence of states $r = s_1, s_2, \cdots$; it is a *run* of TS if $s_1 = s_0$. A *prefix* of a computation is a finite initial segment of it. A computation $r = s_1, s_2, \ldots$ is *safe*, also written $r \subseteq T$, if $s_i \to s_{i+1} \in T$ for every two consecutive states; and $s \xrightarrow{*} s' \subseteq T$ denotes the existence of a safe computation from s to s' (empty if $s = s'$). A class $[t] \in T/\sim$ is *enabled* in a state s if $s \to s' \in [t]$ for some $s' \in S$. A run r is *live* if no class $[t] \in T/\sim$ is enabled in every state of some suffix of r with no member of $[t]$ occurring in the suffix. A run is *correct* if it is safe and live.

A partial equivalence, rather than a total one, is used so that some transitions may carry no liveness requirement at all: only transitions in the domain of $\sim$ form classes and thereby incur a liveness obligation, while transitions outside the domain—belonging to no class—may occur in a correct run but are never required to. A run with uncountably many classes enabled in every state of a suffix is not live.

Lemma A.8 (Extension [26]). *Every finite safe run of a transition system is a prefix of a correct run of it.*

Definition A.9 (Multiagent Transition System [26]). Given agents $P \subset \Pi$ and an arbitrary set S of *states* with a designated *initial state* $s_0 \in S$, a *multiagent transition system* over P and S is a transition system $TS = (C, c_0, T, \sim)$ with *configurations* $C := S^P$, *initial configuration* $c_0 := \{s_0\}^P$, *transitions* $T \subseteq C^2$ a set of transactions over P and S, and $\sim$ a partial equivalence on T.

Rather than specifying a multiagent transition system over a set of agents P directly, it is specified via machine transactions (Definition A.1). A machine transaction over $Q \subseteq P$ defines a set of multiagent transitions over P in which all members of $P \setminus Q$ are stationary:

Definition A.10 (Transaction Closure [26]). Let $P \subset \Pi$, S a set of machine states, and $C := S^P$. For any transition or transaction $t = c \to c'$, we write $t_q := c_q \to c'_q$ and say p is *stationary* in t if $c_p = c'_p$. For a machine transaction $t = (c \to c')$ over S with participants Q, the *P-closure of t*, $t{\uparrow}P$, is the set of transitions over P and S defined by:

$$t{\uparrow}P := \begin{cases} \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge & \text{if } Q \subseteq P \\ \quad \forall p \in P \setminus Q.(p \text{ is stationary in } t')\} & \\ \emptyset & \text{otherwise} \end{cases}$$

If R is a set of machine transactions, each $t \in R$ over some Q and S, then the *P-closure of R*, $R{\uparrow}P$, is the set of transitions over P and S defined by:

$$R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$$

Given a relation $\sim$ on R, its *P-closure* $\sim{\uparrow}P$ is the relation on $R{\uparrow}P$ with $\hat{t}$ $(\sim{\uparrow}P)$ $\hat{t}'$ iff $\hat{t} \in t{\uparrow}P$ and $\hat{t}' \in t'{\uparrow}P$ for some $t \sim t'$. A transition over P is *unary* if it lies in the P-closure of a unary transaction.

Namely, the closure over $P \supseteq Q$ of a machine transaction t over Q includes all transitions t' over P in which members of Q do the same in t and in t', and the rest remain in their current (arbitrary) state. The closure likewise carries any relation on transactions to one on the induced transitions. If distinct transactions in R have disjoint P-closures—as when every participant of every transaction in R changes state—then each transition in $R{\uparrow}P$ has a unique inducing transaction, $\sim{\uparrow}P$ relates two transitions exactly when their inducing transactions are $\sim$-related, and $\sim{\uparrow}P$ is a partial equivalence whenever $\sim$ is.

A set R of volition-guarded transactions with an equivalence $\sim$ on their underlying machine transactions induces a volitional multiagent transition system:

Definition A.11 (Volitional Multiagent Transition System [26]). Given agents $P \subset \Pi$, machine states S with initial state s_0, a set R of volition-guarded transactions and volition-guarded if-then-else transactions—a volition-guarded if-then-else transaction $s \to s'\,;\, s''$ guarded by p standing in R for its then-branch $(s \to s', \{p\})$ together with its else-branch—such that every $(t, Q') \in R$ has the participants of t contained in P, every guard p of a volition-guarded if-then-else transaction is in P, and distinct underlying machine transactions, else-branches included, have disjoint P-closures, and an equivalence $\sim$ on the set $T_R := \{t : (t, Q') \in R \text{ for some } Q'\}$ of underlying machine transactions (a total equivalence, hence in particular a partial equivalence in the sense of Definition A.7), such that no else-branch is in T_R and volition-guarded if-then-else transactions with the same else-branch have $\sim$-equivalent then-branches, the *volitional multiagent transition system induced by* $(S, R, \sim)$ *over* P is the multiagent transition system $(\mathcal{A}^P, c_0, T_V, \sim_V)$ where:

(1) $\mathcal{A} := 2^{T_R/\sim} \times S$ is the *agent state space*;
(2) $c_0 \in \mathcal{A}^P$ is the *initial agent configuration*, with $c_0{}^v_p = \emptyset$ and $c_0{}^m_p = s_0$ for every $p \in P$;
(3) T_V consists of all transitions $e \to e' \in (\mathcal{A}^P)^2$ of one of three forms: (*i*) a *volition change* of some $p \in P$—$e^v_p, e'{}^v_p \subseteq (T_R/\sim)_p$ and $e^v_p \neq e'{}^v_p$, $e^m_p = e'{}^m_p$, and $e_r = e'_r$ for every $r \in P \setminus \{p\}$; (*ii*) a *volitional machine transaction* induced by some volition-guarded transaction $(t, Q') \in R$ per Definition A.4(2); or (*iii*) an *else transaction* induced by some volition-guarded if-then-else transaction in R per Definition A.6;
(4) $\sim_V$ is the restriction of $\sim\!\uparrow P$ (Definition A.10) to the volitional machine transactions and else transactions, relating two of them whenever their inducing machine transactions are $\sim$-equivalent, where the inducing machine transaction of an else transaction is the then-branch $s \to s'$ of its volition-guarded if-then-else transaction, and leaves every volition-change transition outside its domain—so volition changes belong to no class and carry no liveness obligation.

The conditions on else-branches ensure that the machine transactions inducing a transition of T_V are $\sim$-equivalent, so that $\sim_V$ is a partial equivalence. The set R of volition-guarded transactions that specifies a platform thus determines the volitional machine transitions of the induced volitional multiagent transition system, while volition-change transitions are freely available to every agent independent of R.

Definition A.12 (Enabled [26]). Given a set of volition-guarded transactions, each (t, Q') with $t = d \to d'$ a machine transaction over some $Q' \subseteq Q \subseteq P$ and S, and an equivalence $\sim$ on machine transactions: the volition-guarded transaction (t, Q') is *machine-enabled* in agent configuration c over P if $c^m_p = d_p$ for every $p \in Q$, and *enabled* if in addition $[t] \in c^v_q$ for every $q \in Q'$. The else-branch of a volition-guarded if-then-else transaction $s \to s'\,;\, s''$ guarded by p is *enabled* in c if $c^m_p = s$ and $[s \to s'] \notin c^v_p$. An equivalence class $[t]$ is *enabled* in c if some volition-guarded transaction (t', Q') with $t' \in [t]$ is enabled in c, or the else-branch of some volition-guarded if-then-else transaction with then-branch $t' \in [t]$ is enabled in c.

A volition-guarded transaction with an empty guard ($Q' = \emptyset$) requires no volitions and is enabled whenever its machine precondition is met. The class of the then-branch of a volition-guarded if-then-else transaction guarded by p is thus enabled whenever p's machine is in its source state, whether or not p wills it. For the volitional multiagent transition system above, a class of volitional machine transactions and else transactions is enabled at a configuration in the sense of Definition A.7 exactly when some volition-guarded transaction or else-branch inducing it is enabled in the above sense; this determines which runs are live and correct per Definition A.7. Volition-change transitions belong to no class and so impose no liveness obligation; personal choices remain free.

Protocols and grassroots protocols. A protocol is a family of multiagent transition systems, one for each set of agents $P \subset \Pi$, which share an underlying set of local machine states $\mathcal{S}$ with a designated initial state s_0. A *local-states function* maps every set of agents $P \subset \Pi$ to an arbitrary set of local machine states $S(P) \subset \mathcal{S}$ that includes s_0 and satisfies $P \subset P' \subset \Pi \implies S(P) \subset S(P')$.

Definition A.13 (Protocol [26]). A *protocol* $\mathcal{F}$ over a local-states function S is a family of multiagent transition systems that has exactly one transition system $\mathcal{F}(P) = (C(P), c_0(P), T(P), \sim(P))$ for every $P \subset \Pi$, with *agent states* $\mathcal{A}(P)$, configurations $C(P) := \mathcal{A}(P)^P$, initial configuration $c_0(P) \in C(P)$, and partial equivalence $\sim(P)$ on $T(P)$ determined by the protocol, such that $P \subseteq P' \subset \Pi$ implies $\mathcal{A}(P) \subseteq \mathcal{A}(P')$ and $c_0(P)_p = c_0(P')_p$ for every $p \in P$.

Definition A.14 (Interleaving [26]). Let $P, P' \subset \Pi$ be disjoint nonempty sets of agents, $r = c_0, c_1, \ldots$ a run of $\mathcal{F}(P)$, and $r' = d_0, d_1, \ldots$ a run of $\mathcal{F}(P')$. An *interleaving* of r and r' is a sequence $e_0, e_1, \ldots$ of configurations in $C(P \cup P')$ for which there exist non-decreasing sequences of indices $(i_k)_{k\geq 0}$ and $(j_k)_{k\geq 0}$ with $i_0 = j_0 = 0$ such that for every $k \geq 0$:

(1) $(e_k)_p = (c_{i_k})_p$ for every $p \in P$,
(2) $(e_k)_q = (d_{j_k})_q$ for every $q \in P'$,
(3) if e_{k+1} exists, then exactly one of: (*i*) $i_{k+1} = i_k + 1$ and $j_{k+1} = j_k$ (a P-step), or (*ii*) $i_{k+1} = i_k$ and $j_{k+1} = j_k + 1$ (a P'-step).

Moreover, if r is finite of length n then $i_k = n$ for some k, and if r is infinite then for every $m \geq 0$ there is a k with $i_k = m$; likewise for r' and (j_k).

Definition A.15 (Interaction, Interactive Run, First Interaction [26]). Let $Q \subset \Pi$. For a configuration c over Q and $R \subseteq Q$, write $c|_R$ for the restriction of c to R. A transition $e \to e'$ of $\mathcal{F}(Q)$ is an *interaction between* $x \neq y \in Q$ if $e|_{\{x\}} \neq e'|_{\{x\}}$ and $e|_{\{y\}} \neq e'|_{\{y\}}$, and $e|_{\{x\}} \to e'|_{\{x\}}$ is not a transition of $\mathcal{F}(\{x\})$ or $e|_{\{y\}} \to e'|_{\{y\}}$ is not a transition of $\mathcal{F}(\{y\})$. It is an *interaction between* disjoint nonempty $P, P' \subseteq Q$ if it is an interaction between some $x \in P$ and some $y \in P'$. A run $\hat{r}$ of $\mathcal{F}(Q)$ is *interactive between* P *and* P' if some transition of $\hat{r}$ is an interaction between them; the *first interaction* of P and P' in $\hat{r}$ is the first such transition.

Definition A.16 (Oblivious, Interactive, Grassroots [26]). A protocol $\mathcal{F}$ is:

(1) *oblivious* if for every disjoint nonempty $P, P' \subset \Pi$, every interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a correct run of $\mathcal{F}(P \cup P')$.

(2) *interactive* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P' is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.
(3) *grassroots* if it is oblivious and interactive.

Being oblivious means two disjoint groups coexist without interference: any interleaving of a correct run of one with a correct run of the other is a correct run of the combined system. Being interactive means two groups that have not yet interacted can still do so: the combined system has a correct continuation in which a single transaction engages agents of both groups. Agents that have taken part in a common interaction are joined; an instance is a maximal set of agents joined directly or indirectly, and two instances coalesce when they come to be one.

Definition A.17 (Interaction Graph, Instance, Coalescence [26]). Let $\mathcal{F}$ be a protocol, $Q \subset \Pi$, and r a prefix of a run of $\mathcal{F}(Q)$. The *interaction graph* of r has vertices Q and an edge between $a \neq b \in Q$ whenever some transition of r is an interaction between a and b. A nonempty $P \subseteq Q$ is an *instance* in r if it is a connected component of the interaction graph of r. Disjoint nonempty $P, P' \subseteq Q$ *coalesce* in a run $\hat{r}$ of $\mathcal{F}(Q)$ if they are instances in some prefix of $\hat{r}$ and lie in one component of the interaction graph of a longer prefix of $\hat{r}$.

Definition A.18 (Transactions Over a Local-State Function [26]). Let S be a local-states function. A set of transactions R is *over* S if every transaction $t \in R$ is a multiagent transition over Q and $S(P')$ for some $Q \subseteq P' \subset \Pi$. Given such a set R and $P \subset \Pi$, $R(P) := \{t \in R : t \text{ is over } Q \text{ and } S(P'), Q \subseteq P' \subseteq P\}$.

Definition A.19 (Volitional Transactions-Based Protocol [26]). Let S be a local-states function and R a set of volition-guarded transactions over S with equivalence $\sim$. The *protocol $\mathcal{F}$ over R, S, and $\sim$* assigns to each set of agents $P \subset \Pi$ the volitional multiagent transition system $\mathcal{F}(P)$ induced by $(S(P), R(P), \sim)$ over P (Definition A.11). In particular, $C(P) = \mathcal{A}(P)^P$ with agent state space $\mathcal{A}(P) := 2^{T_{R(P)}/\sim} \times S(P)$—monotone in P, as Definition A.13 requires—and $c_0(P)$ has p-component $(\emptyset, s_0)$ for every $p \in P$.

The obliviousness results rest on the following invariant: in a group's own system an agent wills only transactions internal to the group, so the guard of a transaction that reaches another group cannot will it while the group runs alone, and the transaction is never enabled in an interleaving.

Lemma A.20 (Volitional Containment [26]). *Let $\mathcal{F}$ be a volitional transactions-based protocol over a set of volition-guarded transactions R with equivalence $\sim$. For every $P \subset \Pi$, every configuration c of $\mathcal{F}(P)$, and every $p \in P$: $c_p^v \subseteq T_{R(P)}/\sim$, where $T_{R(P)} := \{t : (t, Q') \in R(P)$ for some $Q'\}$.*

Lemma A.21 (Interleaving Safety [26]). *Let $\mathcal{F}$ be a volitional transactions-based protocol and $P, P' \subset \Pi$ disjoint and nonempty. Every finite prefix of an interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a finite safe run of $\mathcal{F}(P \cup P')$.*

Proposition A.22 ([26]). *A volitional transactions-based protocol is oblivious provided that for every disjoint nonempty $P, P' \subset \Pi$, no equivalence class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$.*

Corollary A.23 (Guarded Obliviousness [26]). *A volitional transactions-based protocol over a set of volition-guarded transactions R is oblivious if every volition-guarded transaction $(t, Q') \in R$ whose machine transaction t has two or more participants has a nonempty guard, $Q' \neq \emptyset$.*

In a run of $\mathcal{F}(P \cup P')$ a member may will a transaction reaching the other group, which a group's own run forbids; the volitional notion constrains the resulting coupling:

Definition A.24 (Left [26]). Let $\mathcal{F}$ be a volitional transactions-based protocol over R, $Q \subset \Pi$, c a configuration of $\mathcal{F}(Q)$, and $x \in Q$. Then x *has left* in c if no $(t, Q') \in R(Q)$ with $x \in Q'$ is machine-enabled in any configuration c' of $\mathcal{F}(Q)$ with $c \xrightarrow{*} c' \subseteq T(Q)$.

Definition A.25 (Interactive Among the Agents That Have Not Left [26]). A volitional transactions-based protocol $\mathcal{F}$ is *interactive among the agents that have not left* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', in whose last configuration some agent of P and some agent of P' have not left, is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.

An interactive protocol is interactive among the agents that have not left, the latter requiring a continuation of fewer prefixes. A group all of whose agents have left can never be connected by an act one of them wills, and the definition asks nothing of it.

Definition A.26 (Volitionally Grassroots [26]). A volitional transactions-based protocol $\mathcal{F}$ is *volitionally grassroots* if it is oblivious, interactive among the agents that have not left, and, for every disjoint nonempty $P, P' \subset \Pi$ and every safe run of $\mathcal{F}(P \cup P')$ interactive between P and P', the first interaction of P and P' is induced by a volition-guarded transaction (t, Q') with $Q' \cap P \neq \emptyset$ and $Q' \cap P' \neq \emptyset$.

Two groups thus become connected only by an act a person of each wills, and a grassroots protocol meeting the condition on first interactions is volitionally grassroots. An else transaction (Definition A.6) is over a single participant and so is never an interaction; a binary transaction with an else-branch would let an agent of another group, who never wills it, trigger it, and could be a first interaction willed by neither, which is why Definition A.5 admits only unary ones.

B THE SPEECH-ACT SCHEMAS OF THE CONTRACTS

The speech-act schemas of GAI and GSNP are in the repository, `programs/gai/GAI.glp` and `programs/gai/GSNP.glp` in the GLP repository, each an instance of GnW, `programs/gsc/GnW.glp` [17], with the schemas it replaces and adds, their records `cascade/GAI` and `cascade/GSNP`; the boxes below give the schemas of the instance by the numbers of the clauses they realise, in the notation of Section 3.2. GAI writes *agent* for GnW's *ward*, *principal* and *principal_of* for the two records of a creation, *operator_of* and *operated_by* for the two records of a guardian, at the guardian and at the AI agent, and *retired_agent* for the record of a retirement. A person holds *person* from the act that establishes their identity and an AI agent *agent* from its creation, and a role of a person requires *person*. The schemas of GnW:1 and GnW:2 and those of GnW:3 and GnW:4 for knowing carry the attestation of an AI agent (GAI:2),

and the schemas of GFP:1 and GFP:2 ask that their agent be a person or an AI agent with a guardian; the developer signs the code of an AI agent and the principal the attestation by GFP:1 (GAI:1). GSNP's are GAI's, the schema of GSG:2 also clearing every listing and membership between the two friends (GSN:3), with those of GSN carrying items; a group act is a person's, and an AI agent takes none. $author(x)$ is the first signer of x [17], and a group is the name $\langle Alice \rangle \cdot n$, Alice its leader [17].

GAI: Schemas of GKK

1. $identity(Alice), person(Alice) \leftarrow$
   $\neg identity(Alice)$ guarded by $Alice$.
2. $knows(Alice, Bob), knows(Bob, Alice) \leftarrow$
   $person(Alice), person(Bob)$ guarded by $Alice, Bob$.
3. $knows(Bob, Carol) \leftarrow$
   $knows(Alice, Bob), knows(Alice, Carol), person(Alice), knows(Bob, Alice), person(Bob), person(Carol)$
   guarded by $Alice, Bob$.

GAI: Schemas of GSG

1. $friend(Alice, Bob), friend(Bob, Alice) \leftarrow$
   $knows(Alice, Bob), \neg friend(Alice, Bob), person(Alice), knows(Bob, Alice), \neg friend(Bob, Alice), person(Bob)$
   guarded by $Alice, Bob$.
2. $\neg friend(Alice, Bob), \neg friend(Bob, Alice) \leftarrow$
   $friend(Alice, Bob), person(Alice), \neg operator_of(Alice, Bob), friend(Bob, Alice)$
   guarded by $Alice$.

GAI: Schemas of GnW

1. $principal_of(Alice, Carol), operator_of(Alice, Carol), friend(Alice, Carol), knows(Alice, Carol), agent_of(Alice, Carol, y), identity(Carol), agent(Carol), principal(Carol, Alice), operated_by(Carol, Alice), friend(Carol, Alice), knows(Carol, Alice), attestation(Carol, y) \leftarrow$
   $person(Alice), item(Alice, y), \neg identity(Carol), author(y) = Alice$ guarded by $Alice, Carol$.
2. $operator_of(Bob, Carol), friend(Bob, Carol), knows(Bob, Carol), agent_of(Bob, Carol, y), operated_by(Carol, Bob), friend(Carol, Bob), knows(Carol, Bob) \leftarrow$
   $person(Bob), \neg operator_of(Bob, Carol), person(Alice), operator_of(Alice, Carol), friend(Alice, Bob), agent_of(Alice, Carol, y), agent(Carol), operated_by(Carol, Alice), \neg operated_by(Carol, Bob), \neg friend(Carol, Bob), \forall Erin\,(operated_by(Carol, Erin) \rightarrow identity(Erin) \wedge person(Erin) \wedge operator_of(Erin, Carol))$
   guarded by $Bob, Alice$ and every $Erin$.

   $\neg operator_of(Bob, Carol), \neg operated_by(Carol, Bob) \leftarrow$
   $person(Alice), operator_of(Alice, Carol), person(Bob), operator_of(Bob, Carol), operated_by(Carol, Bob), operated_by(Carol, Alice), \forall Erin \neq Bob\,(operated_by(Carol, Erin) \rightarrow identity(Erin) \wedge person(Erin) \wedge operator_of(Erin, Carol))$
   guarded by $Alice$ and every $Erin$.

   $\neg operator_of(Bob, Carol), \neg operated_by(Carol, Bob) \leftarrow$
   $person(Bob), operator_of(Bob, Carol), operated_by(Carol, Bob)$ guarded by Bob.

   $\neg friend(Bob, Carol), \neg operator_of(Bob, Carol), \neg friend(Carol, Bob), \neg operated_by(Carol, Bob) \leftarrow$
   $person(Bob), friend(Bob, Carol), operator_of(Bob, Carol), friend(Carol, Bob), operated_by(Carol, Bob)$ guarded by Bob.
3. $knows(Carol, Bob), knows(Bob, Carol), agent_of(Bob, Carol, y) \leftarrow$
   $agent(Carol), operated_by(Carol, Alice), attestation(Carol, y), person(Bob)$ guarded by $Carol, Bob$.

   $friend(Carol, Bob), friend(Bob, Carol) \leftarrow$
   $agent(Carol), operated_by(Carol, Alice), knows(Carol, Bob), \neg friend(Carol, Bob), knows(Bob, Carol), person(Bob), \neg friend(Bob, Carol), person(Alice), operator_of(Alice, Carol), friend(Alice, Bob)$ guarded by $Carol, Bob, Alice$.

   $\neg friend(Carol, Bob), \neg friend(Bob, Carol) \leftarrow$
   $friend(Carol, Bob), operated_by(Carol, Alice), \neg operated_by(Carol, Bob), friend(Bob, Carol), person(Alice), operator_of(Alice, Carol)$
   guarded by $Alice$.

   $\neg friend(Carol, Bob), \neg friend(Bob, Carol) \leftarrow$
   $agent(Carol), operated_by(Carol, Alice), friend(Carol, Bob), \neg operated_by(Carol, Bob), friend(Bob, Carol)$ guarded by $Carol$.
4. $knows(Carol, Dave), agent_of(Carol, Dave, z), knows(Dave, Carol), agent_of(Dave, Carol, y) \leftarrow$
   $agent(Carol), operated_by(Carol, Alice), attestation(Carol, y), agent(Dave), operated_by(Dave, Bob), attestation(Dave, z)$
   guarded by $Carol, Dave$.

   $friend(Carol, Dave), friend(Dave, Carol) \leftarrow$
   $agent(Carol), operated_by(Carol, Alice), knows(Carol, Dave), \neg friend(Carol, Dave), agent(Dave), operated_by(Dave, Bob), knows(Dave, Carol), \neg friend(Dave, Carol), person(Alice), operator_of(Alice, Carol), knows(Alice, Dave), friend(Alice, Bob), person(Bob), operator_of(Bob, Dave)$ guarded by $Carol, Dave, Alice, Bob$; $Alice$ and Bob possibly one person.
5. $retired_agent(Carol), \neg friend(Carol, *), \neg operated_by(Carol, *), \forall Bob\,(friend(Carol, Bob) \rightarrow \neg friend(Bob, Carol)), \forall Erin\,(operated_by(Carol, Erin) \rightarrow \neg operator_of(Erin, Carol)) \leftarrow$
   $agent(Carol), operated_by(Carol, Alice), person(Alice), operator_of(Alice, Carol), \forall Bob\,(friend(Carol, Bob) \rightarrow identity(Bob) \wedge friend(Bob, Carol))$,

$\forall Erin\,(operated_by(Carol, Erin) \rightarrow identity(Erin) \wedge person(Erin) \wedge operator_of(Erin, Carol))$
guarded by *Alice* and every *Erin*.

GAI: Schemas of GFP

1. $item(Alice, x) \leftarrow$
   $\neg item(Alice, x)$, $person(Alice) \vee (agent(Alice) \wedge \exists g\ operated_by(Alice, g))$ guarded by *Alice*.
2. $item(Bob, \mathrm{sgn}(x, Alice)) \leftarrow$
   $item(Alice, x)$, $friend(Alice, Bob)$, $person(Alice) \vee (agent(Alice) \wedge \exists g\ operated_by(Alice, g))$
   guarded by *Alice*.

GSNP: Schemas of GSN

1. $item(Bob, x) \leftarrow$
   $item(Alice, x)$, $friend(Alice, Bob)$, $person(Alice) \vee (agent(Alice) \wedge \exists g\ operated_by(Alice, g))$, $author(x) = Alice$ guarded by *Alice*.
2. $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Alice, n) \leftarrow$
   $person(Alice)$, $\neg member(Alice, \langle Alice\rangle \cdot n)$, $\neg closed(Alice, n)$ guarded by *Alice*.

   $closed(Alice, n)$, $\neg listed(Alice, Alice, n)$, $\neg member(Alice, \langle Alice\rangle \cdot n)$, $\forall Bob\,(listed(Alice, Bob, n) \rightarrow \neg member(Bob, \langle Alice\rangle \cdot n)) \leftarrow$
   $person(Alice)$, $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Alice, n)$, $\forall Bob\,(listed(Alice, Bob, n) \rightarrow identity(Bob) \wedge member(Bob, \langle Alice\rangle \cdot n))$
   guarded by *Alice*.
3. $listed(Alice, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $friend(Alice, Bob)$, $person(Alice)$, $person(Bob)$, $\neg member(Bob, \langle Alice\rangle \cdot n)$ guarded by *Alice*, *Bob*.

   $\neg listed(Alice, Bob, n)$, $\neg member(Bob, \langle Alice\rangle \cdot n) \leftarrow$
   $listed(Alice, Bob, n)$, $person(Bob)$, $member(Bob, \langle Alice\rangle \cdot n)$ guarded by *Bob*.

   $\neg listed(Alice, Bob, n)$, $\neg member(Bob, \langle Alice\rangle \cdot n) \leftarrow$
   $person(Alice)$, $listed(Alice, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n)$ guarded by *Alice*.

   $\neg friend(Alice, Bob)$, $\neg friend(Bob, Alice)$, $\neg listed(Alice, Bob, *)$, $\neg member(Alice, \langle Bob\rangle \cdot *)$, $\neg listed(Bob, Alice, *)$, $\neg member(Bob, \langle Alice\rangle \cdot *) \leftarrow$
   $friend(Alice, Bob)$, $person(Alice)$, $\neg operator_of(Alice, Bob)$, $friend(Bob, Alice)$
   guarded by *Alice*.
4. $posted(Alice, x, n) \leftarrow$
   $person(Alice)$, $item(Alice, x)$, $member(Alice, \langle Alice\rangle \cdot n)$, $\neg posted(Alice, x, n)$, $author(x) = Alice$ guarded by *Alice*.

   $posted(Alice, x, n) \leftarrow$
   $person(Bob)$, $item(Bob, x)$, $member(Bob, \langle Alice\rangle \cdot n)$, $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Bob, n)$, $friend(Alice, Bob)$, $\neg posted(Alice, x, n)$, $author(x) = Bob$ guarded by *Bob*.

   $posted(Alice, \mathrm{sgn}(x, Bob), n) \leftarrow$
   $person(Bob)$, $item(Bob, x)$, $member(Bob, \langle Alice\rangle \cdot n)$, $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Bob, n)$, $friend(Alice, Bob)$ guarded by *Bob*.

   $delivered(Alice, y, Bob, n)$, $item(Bob, \mathrm{sgn}(y, Alice)) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $posted(Alice, y, n)$, $listed(Alice, Bob, n)$, $friend(Alice, Bob)$, $\neg delivered(Alice, y, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n)$, $author(y) \neq Alice$ guarded by no one.

   $delivered(Alice, y, Bob, n)$, $item(Bob, y) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $posted(Alice, y, n)$, $listed(Alice, Bob, n)$, $friend(Alice, Bob)$, $\neg delivered(Alice, y, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n)$, $author(y) = Alice$ guarded by no one.

   $delivered(Alice, y, Alice, n)$, $item(Alice, y) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $posted(Alice, y, n)$, $\neg delivered(Alice, y, Alice, n)$, $author(y) \neq Alice$ guarded by no one.

We name the schemas as the repository does: `establish`, `acquaint` and `introduce` (GKK:1 to GKK:3); `befriend` and `unfriend` (GSG:1 and GSG:2); `enter` (GnW:1); `make_guardian`, `end_guardian`, `resign` and `guardian_unfriend` (GnW:2); `agent_acquaint`, `agent_befriend`, `agent_unfriend` and `agent_end` (GnW:3); `agents_acquaint` and `agents_befriend` (GnW:4); `retire` (GnW:5); `sign` and `forward` (GFP:1 and GFP:2); and `message` (GSN:1), `create` and `close` (GSN:2), `join`, `leave` and `remove` (GSN:3), and `post_own`, `post`, `post_forward`, `deliver`, `deliver_own` and `deliver_leader` (GSN:4).

The schema of GnW:5 takes every friend Bob and every guardian Erin of the AI agent by its quantified conjuncts, and clears *friend* and *operated_by* at it, and the first two schemas of GnW:2 take every guardian Erin likewise [17, 22]; Alice, the guardian an AI agent holds recorded in its acts of knowing and in ending a friendship, is an agent variable there and takes no part in the act. The predicate *knows* is a multiset: the acts of GKK:2, GnW:3 and GnW:4 for knowing neither require nor forbid that the two know each other, and add a copy of each at the other whether they do or not, so that two agents who know each other may take the act again. *knows* and *friend* are reflexive, so that in the act of GnW:4 for a relation one person may be the guardian of both AI agents.

Transaction equivalence. The equivalence $\sim$ is that of [17]: one schema at one admissible binding determines one class, whose members differ only in the configuration in which they occur.

C THE CONTRACTS CERTIFIED

Contracts with wards, agents who act only while they have a guardian, are defined in [17], with conditions on the schemas of a contract, decided by the checker in the repository https://github.com/EShapiro2/GLP: a contract has *kinds*, a mark for a person and a mark for each kind of ward, held from the act that establishes an agent's identity; *guardianships*, the two records of a guardian, at the guardian and at the ward, with every guarding role a person's,

Table 1: The predicates of the contract; the first argument of each is its holder p.

predicate	meaning
$identity(p)$	p has a genuine digital identity
$person(p)$	p is a person
$agent(p)$	p is an AI agent
$knows(p, q)$, a multiset	p knows q, by q's public key
$friend(p, q)$	q is a friend of p
$principal(p, q)$, $principal_of(p, q)$	q is the principal of p; q is an AI agent p created
$operator_of(p, q)$, $operated_by(p, q)$	p is a guardian of the AI agent q; q is a guardian of the AI agent p
$attestation(p, y)$	y is the attestation of the AI agent p
$agent_of(p, q, y)$	p holds recorded the AI agent q with its attestation y
$retired_agent(p)$	the AI agent p was retired
$item(p, x)$, a multiset	p holds the item x, signed by its author and by every agent that forwarded it, in order

Table 2: The atoms the acts of the example add and delete.

Acts	Alice	Bob	Alf
GKK:1, by Alice and by Bob	*identity*, *person*	*identity*, *person*	
GKK:2	$knows(Bob)$	$knows(Alice)$	
GSG:1	$friend(Bob)$	$friend(Alice)$	
GFP:1, Alice signing y	$item(y)$		
GnW:1	$principal_of(Alf)$, $operator_of(Alf)$, $friend(Alf)$, $knows(Alf)$, $agent_of(Alf, y)$		*identity*, *agent*, $principal(Alice)$, $operated_by(Alice)$, $friend(Alice)$, $knows(Alice)$, $attestation(y)$
GnW:3, knowing		$knows(Alf)$, $agent_of(Alf, y)$	$knows(Bob)$
GnW:3, a relation		$friend(Alf)$	$friend(Bob)$
GFP:1 and GSN:1, Alf signing and sending x		$item(x)$	$item(x)$
GnW:5	$\neg operator_of(Alf)$, $\neg friend(Alf)$	$\neg friend(Alf)$	$retired_agent$, $\neg friend(*)$, $\neg operated_by(*)$

a ward's who has a guardian, or that of an agent who *can act*, a person or a ward with a guardian; an introductory act *unobstructed among the free*, by which two persons come to know each other; and *relay acts*, by which a ward with a guardian and another agent come to know each other, the ward presenting what it carries and the other recording it. A contract with these has an *introduction with wards*, and the protocol realising a contract with an introduction with wards, traceable provenance and volition is volitionally grassroots (Theorem Grassroots realisation with wards of [17]). GnW is certified so there, and GAI and GSNP are certified here, on the schemas of Appendix B.

Proposition C.1 (Grassroots). *The protocol realising GAI is volitionally grassroots.*

Proof. GAI has the schemas of an instance of GnW but for `enter`, `make_guardian`, `agent_acquaint` and `agents_acquaint`, which have the roles, the guards and the atoms of GnW's and the attestation besides, and `sign` and `forward`; the instance has an introduction with wards, traceable provenance and volition [17], and we check what the six change. *Kinds.* The kinds are *identity*, *person* and {*agent*}: no schema deletes any of the three, `establish` adds *identity* with *person* and `enter` adds it at the AI agent with *agent*, each forbidding it there, and no other schema adds one of them. *Guardianships.* The guardianship of *agent* has *operator_of* and *operated_by* for its two records, which `enter` and `make_guardian` add as GnW's do and the other four neither add, delete nor clear; every guarding role of the instance forbids *identity*, requires *person*, or requires *agent* and an atom of *operated_by*, the six among them but for the role of `sign` and the first role of `forward`, whose bodies hold that their agent can act. *Introduction.* `acquaint` is unobstructed among the free, as in GnW. `agent_acquaint` is a relay act with the ward role Carol and the free role Bob, and `agents_acquaint` one with the ward roles Carol and Dave, neither with a guardian role and neither forbidding anything: a ward role requires *identity*, *agent*, *operated_by* of an agent variable occurring nowhere else, and $attestation(y)$, with y a speech-act variable occurring in no other atom required, which the definition of a relay act admits as what a ward carries, since `enter`, the one schema adding *agent*, adds $attestation(y)$ at the AI agent and its record $agent_of(r, y)$ at the principal, where it adds $operator_of(r)$, `make_guardian`, the other schema adding $operator_of(r)$, requires the record at a guardian of r and adds it at the new guardian, and no schema deletes an atom of *attestation* or of *agent_of*; the free role requires *identity* and *person*; and `agent_acquaint` adds the record at its free role, and `agents_acquaint` the record of each ward role at the other. *Traceable provenance.* Every atom a schema adds has for its arguments roles of the schema and speech-act variables: `enter` adds $attestation(y)$ and $agent_of(r, y)$ with $item(y)$ required at the principal; `make_guardian`, `agent_acquaint` and `agents_acquaint` add $agent_of(r, y)$ with $agent_of(r, y)$ or $attestation(y)$ required at a role; `sign` adds $item(x)$, x required at no role; and `forward` adds $item(\mathrm{sgn}(x, Alice))$, whose arguments are the role Alice of the signed term and x, which the required $item(x)$ carries. *Volition.* The four schemas that replace GnW's have its roles, its guards and the atoms it requires; `sign` has arity one; and the role Alice of `forward` requires $friend(Bob)$. So GAI has an introduction with wards, traceable provenance and volition, and the protocol realising it is volitionally grassroots by Theorem Grassroots realisation with wards of [17]. □

Proposition C.2 (The social network with provenance is grassroots). *The protocol realising GSNP is volitionally grassroots.*

Proof. GSNP has the schemas of GAI, `unfriend` also clearing every listing and membership between the two, with `message` and the schemas of a group, which name a group by its leader and a number and close it by a listed role, the leader with every member the leader lists [17]. It has the kinds, the guardianship, the introductory act and the relay acts of GAI, as in the proof of Proposition C.1: the added schemas add and delete none of *identity*, *person*, *agent*, *operator_of*, *operated_by*, *attestation* and *agent_of*;

the role of `message` holds in its body that its agent can act; every guarding role of a schema of a group requires *person*; and `deliver`, `deliver_own` and `deliver_leader` have no guard. The set of all the predicates has traceable provenance: the schemas of GAI as in that proof; `message` adds $item(x)$ at Bob, whose speech-act variable the required $item(x)$ at Alice carries; and of the schemas of a group, `create` adds atoms whose arguments are its role, the name based on it and a variable it requires nowhere, `join`, `close`, `post`, `post_own`, `post_forward`, `deliver`, `deliver_own` and `deliver_leader` add atoms whose arguments are roles, the roles of signed terms, that name and variables an agent role of the schema requires, and `leave`, `remove` and `unfriend` add nothing. Volition holds: the schemas of GAI as in that proof; the role Alice of `message` and of `unfriend` requires $friend(Bob)$, an atom of traceable provenance naming the other role; `join` is guarded in both its roles, the two roles of `close` have the base Alice, `create`, `post_own` and `deliver_leader` have arity one, and the role Alice of `leave`, `remove`, `deliver` and `deliver_own` requires $listed(Bob, n)$ and the role Bob of `post` and `post_forward` requires $member(\langle Alice \rangle \cdot n)$, each an atom of traceable provenance an argument of which has the base of the other role. So GSNP has an introduction with wards, traceable provenance and volition, and the protocol realising it is volitionally grassroots by Theorem Grassroots realisation with wards of [17]. □